

Exhaustive Production Code Review

Reviewed 2026-08-02 against the repository as supplied. The review covered every tracked Kotlin production and test file in `:app`, `:core-messaging`, `:protection-engine`, and `:design-system`, plus manifests, resources, Room schemas and migrations, Gradle/build logic, dependency verification, CI/release scripts, operational documentation, and reachable Git history. Findings below are root-cause deduplicated; related locations are listed together rather than reporting the same defect in several categories.

A filesystem inventory found no `Codex_final_review.mdat` or other `.mdat` artifact in the supplied repository, so no conclusion was inferred from that suspicious-looking name.

This was a static review. Existing tests and CI definitions were inspected, but Gradle tasks were not rerun because they create build/cache artifacts and the request permits creation of only this report file. “Critical” means direct loss of the feature’s core security or data guarantee; “High” means likely data loss, duplicate transmission, security bypass, or major policy exposure; “Medium” means a bounded but user-visible correctness/security failure; “Low” means a smell, hardening gap, or limited-impact defect.

Security Issues

S-1. Locked-space encryption deliberately fails open to plaintext

- **Severity:** Critical
- **Location:**
`core-messaging/src/main/kotlin/com/messages/core/secret/LockedContent.kt:120`,
`core-messaging/src/main/kotlin/com/messages/core/secret/LockedContent.kt:253`,
`core-messaging/src/test/kotlin/com/messages/core/secret/LockedContentTest.kt:129`
- **What is wrong:** `sealText` returns its plaintext input whenever the content key cannot be obtained or any cipher operation throws. The test suite explicitly requires this fail-open behavior.
- **How it fails:** Invalidate or temporarily make Android Keystore unavailable, then receive a message routed to a locked conversation. The Telephony copy can be purged while the Room `body` and `normalizedBody` are committed unchanged, so a database extraction reveals the supposedly locked message without the PIN.
- **Suggested fix:** Never commit a locked row as plaintext. Keep the provider copy until encryption succeeds, or store the arrival in a small fail-closed retry journal encrypted by a separately available key; surface a persistent user warning and retry. Change the regression test to require that no locked plaintext reaches Room.

S-2. The ciphertext marker is also an encryption-bypass prefix

- **Severity:** High

- **Location:**

core-messaging/src/main/kotlin/com/messages/core/secret/LockedContent.kt:116,
core-messaging/src/main/kotlin/com/messages/core/secret/LockedContent.kt:120,
core-messaging/src/main/kotlin/com/messages/core/secret/LockedContent.kt:139

- **What is wrong:** Any input beginning with the literal `\u0001lc1:` marker is assumed already sealed, without validating that it is a valid ciphertext.
- **How it fails:** An SMS body beginning with that control character and marker is routed into locked space. `sealText` returns it unchanged; later `openText` fails Base64/GCM validation and again returns it unchanged. The attacker-controlled plaintext remains readable in Room indefinitely while `isCanonical` treats it as encrypted.
- **Suggested fix:** Store encryption state in a schema column and validate/decrypt before treating a value as sealed. At minimum, if a marker-prefixed value does not authenticate, encrypt the entire original string as plaintext instead of accepting it as canonical.

S-3. Restoring locked trash republishes its plaintext to the system SMS database

- **Severity:** High

- **Location:**

core-messaging/src/main/kotlin/com/messages/core/MessageRepository.kt:1668,
core-messaging/src/main/kotlin/com/messages/core/MessageRepository.kt:1690,
core-messaging/src/main/kotlin/com/messages/core/MessageRepository.kt:1741

- **What is wrong:** `restoreFromTrash` opens a locked row, passes the opened body to `restoreIncomingProviderRow/writeOutgoingSmsToProvider`, and only reseals the Room row. It does not purge the newly created provider row for locked space.
- **How it fails:** Delete a locked SMS, then restore it from locked trash. A plaintext `content://sms` row is inserted and mapped; it remains visible to another future default-SMS app and forensic tools even after the Messages UI is locked.
- **Suggested fix:** Never restore a locked row to Telephony. Restore only the encrypted Room row/local media while it remains in locked space, and add an integration test asserting that provider insertion is not called for `Spaces.LOCKED`.

S-4. Chat OTP copy bypasses the hardened sensitive-clipboard path

- **Severity:** Medium

- **Location:** `app/src/main/kotlin/com/messages/app/ui/chat/ChatScreen.kt:2056`,
`app/src/main/kotlin/com/messages/app/ui/chat/ChatScreen.kt:2060`,
`app/src/main/kotlin/com/messages/app/receiver/OtherReceivers.kt:361`
- **What is wrong:** Notification OTP copy uses `OtpClipboard`, but the in-chat chip calls `Compose clipboard` directly, omitting sensitive marking/expiry behavior.
- **How it fails:** Copy an OTP from the chat chip rather than its notification. The code remains in ordinary clipboard history and is available to keyboards/clipboard readers longer than the protected path, despite identical UI purpose.
- **Suggested fix:** Route every OTP copy action through one hardened helper, mark the clip sensitive, expire only the matching value, and test both entry points.

Cryptography & Key Management Issues

C-1. GCM AAD permits authenticated ciphertext swaps between rows

- **Severity:** High
- **Location:**
`core-messaging/src/main/kotlin/com/messages/core/secret/LockedContent.kt:95`,
`core-messaging/src/main/kotlin/com/messages/core/secret/LockedContent.kt:120`,
`core-messaging/src/main/kotlin/com/messages/core/secret/LockedContent.kt:139`
- **What is wrong:** AAD contains only the field name (`body`, `normalizedBody`, or `lastMessage`), not a stable row/conversation identifier, space, schema version, or message metadata.
- **How it fails:** Someone able to modify a copied Room database can swap two locked `body` ciphertexts. Both still authenticate under AAD `body`, so the app displays message B as if sender/thread/timestamp A authored it, corrupting attribution without an authentication failure.
- **Suggested fix:** Bind ciphertext to a stable immutable record ID, field, space, and format version in AAD. Introduce a versioned envelope and migrate existing rows after assigning stable IDs.

C-2. The locked content key is software-unwrapped and retained in heap

- **Severity:** Medium
- **Location:**
`core-messaging/src/main/kotlin/com/messages/core/secret/LockedContent.kt:25`,
`core-messaging/src/main/kotlin/com/messages/core/secret/LockedContent.kt:110`,
`core-messaging/src/main/kotlin/com/messages/core/secret/LockedContent.kt:253`
- **What is wrong:** Contrary to the comment that the content key “never leaves the Android Keystore in the clear,” Keystore only wraps a 32-byte software key; the raw key is returned as a

`ByteArray` and cached for the process lifetime.

- **How it fails:** A heap dump, in-process memory disclosure, or code execution under the app UID recovers the one key that decrypts every locked body, with no need to know the PIN. The cached array is not cleared on ordinary lock/background transitions.
- **Suggested fix:** Correct the threat-model documentation immediately. Prefer direct operations with a non-exportable Keystore key; if backup portability requires a software content key, unwrap it only for an authenticated session, minimize lifetime, overwrite buffers on lock, and use hardware-backed/user-authentication-bound wrapping where available.

C-3. One unauthenticated Keystore alias serves two distinct key domains

- **Severity:** Medium
- **Location:**
`core-messaging/src/main/kotlin/com/messages/core/secret/LocalKeyBox.kt:61,`
`core-messaging/src/main/kotlin/com/messages/core/secret/LocalKeyBox.kt:95,`
`core-messaging/src/main/kotlin/com/messages/core/secret/SecretSpace.kt:128,`
`core-messaging/src/main/kotlin/com/messages/core/secret/LockedContent.kt:262`
- **What is wrong:** The same `secret_space_kek` AES key, with no `setUserAuthenticationRequired`, wraps both the credential-derived backup KEK and the unrelated message-content key, and the wrapped blobs have no domain AAD.
- **How it fails:** Copying one valid 32-byte wrapped value into the other preference slot passes GCM and length validation. The content-key path then accepts the backup KEK as its message key, making existing messages opaque and allowing the plaintext fail-open path on subsequent writes; the PIN never participates in Keystore authorization.
- **Suggested fix:** Use separate aliases and domain-specific AAD for the backup KEK and content key. Decide explicitly whether locked-space decryption should require recent user authentication, and configure Keystore authentication parameters accordingly.

C-4. Key destruction reports success even if durable deletion fails

- **Severity:** Medium
- **Location:**
`core-messaging/src/main/kotlin/com/messages/core/secret/LockedContent.kt:286,`
`core-messaging/src/main/kotlin/com/messages/core/secret/LockedContent.kt:291`
- **What is wrong:** `destroyKey` ignores the Boolean returned by synchronous `SharedPreferences.commit()`.

- **How it fails:** With a full/corrupt filesystem, secret-space reset clears only the in-memory cache while the wrapped key remains on disk. Recoverable database pages or media indexed under the old key can still be decrypted, despite the reset flow claiming cryptographic erasure.
- **Suggested fix:** Check `commit()`, retry or abort reset on failure, verify the preference is absent, and delete/rotate the Keystore alias only after all dependent data and durable wrappers are removed.

Drive Key Custody Issues

C-5. Drive password protection accepts a one-character password

- **Severity:** High
- **Location:** `app/src/main/kotlin/com/messages/app/ui/drivebackup/DriveKeyProtection.kt:282`, `core-messaging/src/main/kotlin/com/messages/core/backup/MasterKeyVault.kt:179`
- **What is wrong:** Validation rejects only blank input; a single-character password is accepted to wrap the Drive master key.
- **How it fails:** A copied encrypted backup plus its password-wrapped key gives an offline attacker a tiny candidate space. A password such as `a` is recovered despite PBKDF2, exposing the entire SMS/MMS snapshot.
- **Suggested fix:** Enforce a length/strength floor suitable for offline encryption, reject known-compromised choices where feasible, show an entropy-oriented strength meter, and support a generated recovery phrase as the recommended path.

C-6. Disabling user-held custody trusts an existing cloud key without comparison

- **Severity:** High
- **Location:** `app/src/main/kotlin/com/messages/app/drive/DriveBackup.kt:493`, `app/src/main/kotlin/com/messages/app/drive/DriveBackup.kt:506`
- **What is wrong:** The transition back to app-managed custody accepts an existing plaintext key file as authoritative and deletes the protected vault without first proving that both contain the same master key.
- **How it fails:** If a stale or foreign plaintext key remains in `appDataFolder`, disabling user custody removes the only wrapper for the current key. Future backups use the other key and existing encrypted snapshots become permanently undecryptable.

- **Suggested fix:** Decrypt both representations, compare constant-time, and upload/verify the current key before deleting the vault. Make the transition transactional and retain the old wrapper until a round-trip restore succeeds.

C-7. Recovery material is copied to an indefinitely readable clipboard

- **Severity:** Medium
- **Location:**
`app/src/main/kotlin/com/messages/app/ui/drivebackup/DriveKeyProtection.kt:253,`
`app/src/main/kotlin/com/messages/app/ui/drivebackup/DriveKeyProtection.kt:270`
- **What is wrong:** The recovery code is placed on the global clipboard without marking it sensitive or scheduling removal.
- **How it fails:** After the user copies the code, a keyboard/clipboard-history service or a person using the device later can retrieve the credential and decrypt backups acquired from Drive.
- **Suggested fix:** On supported APIs mark `ClipDescription.EXTRA_IS_SENSITIVE`, clear only the matching clip after a short timeout, warn about clipboard history, and favor direct save/share to a user-selected secure destination.

Data Storage & Privacy Issues

D-1. Locked-space attachments remain plaintext

- **Severity:** High
- **Location:**
`core-messaging/src/main/kotlin/com/messages/core/secret/LockedContent.kt:52,`
`core-messaging/src/main/kotlin/com/messages/core/MessageRepository.kt:625`
- **What is wrong:** Locked encryption covers text columns only; `mediaUri` points to an ordinary app-private attachment file.
- **How it fails:** Lock a chat containing an ID photo, audio message, or video, then extract app-private storage from a rooted/debuggable/forensic image. The message text is ciphertext but the attachment opens directly without the secret-space credential.
- **Suggested fix:** Encrypt attachment streams with per-file keys wrapped by the locked content key, authenticate MIME/name/message ID as AAD, decrypt through a short-lived internal provider/stream, and securely migrate/delete old plaintext files.

D-2. Locked-space correspondent and behavioral metadata are deliberately exposed

- **Severity:** Medium
- **Location:**
`core-messaging/src/main/kotlin/com/messages/core/secret/LockedContent.kt:52,`
`core-messaging/src/main/kotlin/com/messages/core/secret/LockedContent.kt:59`
- **What is wrong:** Addresses, timestamps, categories, read/sent state, protection labels, and attachment metadata remain cleartext. This is documented in code, but materially narrows what “private/locked space” protects.
- **How it fails:** A database copy reveals who is in the locked space, when every exchange occurred, whether it was read, and whether it was classified as financial/fraud-related even when bodies decrypt correctly only in-app.
- **Suggested fix:** Make this limitation prominent in product/privacy copy. Longer term, split an encrypted private-store schema from the normal searchable index and retain only keyed routing tokens plus the minimum scheduling metadata in cleartext.

D-3. Secret credentials are retained in immutable Compose state objects

- **Severity:** Medium
- **Location:** `app/src/main/kotlin/com/messages/app/ui/secret/SecretInputs.kt:546,`
`app/src/main/kotlin/com/messages/app/ui/secret/SecretInputs.kt:668,`
`app/src/main/kotlin/com/messages/app/ui/secret/SecretSettingsScreen.kt:182`
- **What is wrong:** PINs/pattern material are converted to immutable `String/array`-backed Compose state and survive recompositions; callers cannot reliably overwrite those objects after use.
- **How it fails:** After setup, unlock, or credential change, a process heap snapshot can retain old and new credential copies until garbage collection, extending exposure far beyond the cryptographic operation.
- **Suggested fix:** Keep credentials in mutable `CharArray/byte` buffers owned by a short-lived controller, avoid conversion to `String`, overwrite immediately after derivation, and clear UI state on lifecycle stop and every terminal result.

D-4. Debug cleanup logs message content fragments and senders

- **Severity:** Low

- **Location:**

`app/src/debug/kotlin/com/messages/app/debug/CleanupTestResidueReceiver.kt:75`

- **What is wrong:** The debug-only cleanup receiver writes sender/body fragments to Logcat.
- **How it fails:** A QA/debug build processing real messages leaves PII in system logs that can be collected with bug reports or by attached development tooling.
- **Suggested fix:** Log only opaque row IDs/counts, gate any detailed diagnostics behind an explicit local flag, and add a static test forbidding body/address values in all build variants.

Concurrency & Threading Issues

T-1. Moving a conversation between spaces is not atomic

- **Severity:** High

- **Location:**

`core-messaging/src/main/kotlin/com/messages/core/MessageRepository.kt:1500,`
`core-messaging/src/main/kotlin/com/messages/core/MessageRepository.kt:1503,`
`core-messaging/src/main/kotlin/com/messages/core/MessageRepository.kt:1507`

- **What is wrong:** Message space update, body recoding, provider purge, conversation merge/delete, and summary refresh are separate operations outside a Room transaction.
- **How it fails:** Kill the process after `setThreadSpace` but before `recodeThread`: rows now claim LOCKED while their bodies are plaintext. Kill it later before the conversation row moves and the messages can become invisible/orphaned. Startup repair scans locked content only, so interruption while unlocking is not symmetrically repaired.
- **Suggested fix:** Put all Room mutations in one `withTransaction`; model external provider purge as a durable outbox state and resume it idempotently. Add a bidirectional startup invariant repair and process-death fault-injection tests at every boundary.

T-2. Secret-space wipe races message intake and can strand undecryptable rows

- **Severity:** High

- **Location:**

`core-messaging/src/main/kotlin/com/messages/core/MessageRepository.kt:1475,`
`core-messaging/src/main/kotlin/com/messages/core/MessageRepository.kt:1485`

- **What is wrong:** Wipe enumerates/deletes rows and then destroys the key without excluding concurrent SMS/MMS intake or locked-space moves.

- **How it fails:** An incoming locked message can be inserted after `allInSpace` is read but before `destroyKey`. It survives the deletion loop encrypted under the destroyed key, becoming permanently opaque; a later intake may also recreate routing/content with inconsistent keys.
- **Suggested fix:** Serialize wipe, intake routing, and space moves with a process-wide mutex plus a durable “wipe in progress” state; perform database deletion in a transaction and make intake defer/retry until key destruction completes.

T-3. SMS logical deduplication is a check-then-insert race

- **Severity:** High
- **Location:**
`core-messaging/src/main/kotlin/com/messages/core/MessageRepository.kt:181,`
`core-messaging/src/main/kotlin/com/messages/core/MessageRepository.kt:186,`
`core-messaging/src/main/kotlin/com/messages/core/MessageRepository.kt:249`
- **What is wrong:** Sender/timestamp/body duplicate detection is not protected by a unique key or transaction.
- **How it fails:** Two concurrent redeliveries of the same `SMS_DELIVER` both observe no row, each creates a Telephony row and Room row, increments unread count twice, and can produce two notifications.
- **Suggested fix:** Persist a canonical logical-delivery fingerprint with a unique index and use insert-or-ignore in one transaction; reconcile/remove any provider row created by the losing insert.

T-4. Startup can release a scheduled-send claim while its worker is running

- **Severity:** High
- **Location:** `app/src/main/kotlin/com/messages/app/MessagesApp.kt:71,`
`app/src/main/kotlin/com/messages/app/MessagesApp.kt:82,`
`core-messaging/src/main/kotlin/com/messages/core/db/MessagesDatabase.kt:171`
- **What is wrong:** Application startup asynchronously resets every `CLAIMED` row while `WorkManager` is free to start workers against the same database.
- **How it fails:** A worker claims a scheduled message, then the startup coroutine resets it to pending before radio dispatch. A second worker can claim and transmit the same message, producing duplicate SMS/MMS sends.
- **Suggested fix:** Include lease owner/generation and expiry in each claim, recover only expired leases atomically, and initialize/reconcile the scheduler before allowing workers to execute.

T-5. Conversation unread counts and previews use a lost-update read-modify-write

- **Severity:** Medium
- **Location:**
`core-messaging/src/main/kotlin/com/messages/core/MessageRepository.kt:1300,`
`core-messaging/src/main/kotlin/com/messages/core/MessageRepository.kt:1309,`
`core-messaging/src/main/kotlin/com/messages/core/MessageRepository.kt:1319`
- **What is wrong:** Concurrent arrivals read the same conversation and each upserts a complete replacement row.
- **How it fails:** Two messages for one thread arrive together; both read unread count 4 and write 5, losing one unread. Whichever upsert finishes last can also replace the preview/category chosen by the newer timestamp computation in the other coroutine.
- **Suggested fix:** Use transactional SQL updates (`unreadCount = unreadCount + 1`) and a timestamp-guarded preview update, or serialize per `(threadId, space)` and keep a database constraint/invariant test.

T-6. MMS transaction consumption is non-atomic

- **Severity:** Medium
- **Location:** `app/src/main/kotlin/com/messages/app/mms/MmsTransactions.kt:106,`
`app/src/main/kotlin/com/messages/app/mms/MmsTransactions.kt:109`
- **What is wrong:** `consume` reads a token and removes it in two separate `SharedPreferences` operations, and ignores removal failure.
- **How it fails:** Two callbacks on different receiver instances can both read the record before either removal commits, so both parse/store/notify the same MMS. If `commit()` fails, later replays continue to authorize the token.
- **Suggested fix:** Move transaction records to Room with an atomic `UPDATE ... WHERE state = PENDING` claim, unique token, and checked durable settlement.

T-7. Overlapping headless replies can stop each other's service

- **Severity:** Medium
- **Location:**
`app/src/main/kotlin/com/messages/app/service/HeadlessSmsSendService.kt:27,`
`app/src/main/kotlin/com/messages/app/service/HeadlessSmsSendService.kt:57,`
`app/src/main/kotlin/com/messages/app/service/HeadlessSmsSendService.kt:70`

- **What is wrong:** Each start launches work but calls `stopSelf()` rather than `stopSelfResult(startId)` when that individual job finishes.
- **How it fails:** Start A is slow; start B arrives and completes first. B calls `stopSelf`, destroys the service scope, and cancels A before its reply is durably sent.
- **Suggested fix:** Track `startId`, call `stopSelfResult(startId)`, and use a queue/supervised scope whose lifetime ends only after all earlier starts settle.

T-8. Sender reputation updates lose concurrent evidence

- **Severity:** Medium
- **Location:**
`core-messaging/src/main/kotlin/com/messages/core/MessageRepository.kt:1967,`
`core-messaging/src/main/kotlin/com/messages/core/MessageRepository.kt:1973`
- **What is wrong:** Reputation is read, modified in memory, and upserted without an atomic increment/update.
- **How it fails:** Two simultaneous classification outcomes for the same normalized sender both read count 10 and write 11, so one fraud/spam/user correction is lost and future classification uses understated evidence.
- **Suggested fix:** Express counters as atomic SQL increments in a transaction and make last-updated/max severity updates conflict-safe.

T-9. Dashboard period changes race and can render stale data

- **Severity:** Medium
- **Location:**
`app/src/main/kotlin/com/messages/app/ui/dashboard/DashboardScreen.kt:63,`
`app/src/main/kotlin/com/messages/app/ui/dashboard/DashboardScreen.kt:85`
- **What is wrong:** Each period selection starts independent asynchronous loading without cancelling or version-checking the previous job.
- **How it fails:** Select 30 days and immediately 7 days; if the 30-day query finishes last, its result overwrites the 7-day selection and labels, displaying the wrong analytics.
- **Suggested fix:** Represent the period as a `StateFlow` and load with `flatMapLatest`, or attach a request generation and discard stale completions.

T-10. Local backup import/export operations can overlap

- **Severity:** Medium

- **Location:**

app/src/main/kotlin/com/messages/app/ui/settings/SettingsScreen.kt:349,
app/src/main/kotlin/com/messages/app/ui/settings/SettingsScreen.kt:907,
core-messaging/src/main/kotlin/com/messages/core/backup/LocalArchive.kt:90

- **What is wrong:** UI actions launch archive operations without a shared operation mutex or durable operation state.
- **How it fails:** Double-tap export/import or rotate/re-enter while work is active; two restores can mutate Room/preferences concurrently, or two writers can target the same document descriptor, producing partial results and misleading completion state.
- **Suggested fix:** Centralize archive work in a single lifecycle-independent coordinator, reject/queue concurrent operations, use unique operation IDs, and disable all conflicting actions until durable completion.

T-11. Outbox cancellation reverses the safe DB/WorkManager ordering

- **Severity:** Medium
- **Location:** app/src/main/kotlin/com/messages/app/ui/outbox/OutboxScreen.kt:166,
app/src/main/kotlin/com/messages/app/ui/outbox/OutboxScreen.kt:200
- **What is wrong:** The screen cancels WorkManager before making the durable database transition that says the scheduled message is cancelled/deleted.
- **How it fails:** If cancellation succeeds and the following DB operation throws or the process dies, the row still says `SCHEDULED` but no worker remains to send it. Conversely, a worker already starting can transmit before the row transition wins.
- **Suggested fix:** Atomically compare-and-set the row to `CANCELLED` first; workers must claim only `SCHEDULED`. Then cancel unique work as cleanup and reconcile cancelled/orphaned work at startup.

Messaging Correctness & Logic Issues

L-1. A partial group-SMS dispatch is retried as a complete duplicate

- **Severity:** High
- **Location:** app/src/main/kotlin/com/messages/app/schedule/Scheduling.kt:35,
app/src/main/kotlin/com/messages/app/schedule/Scheduling.kt:51,
app/src/main/kotlin/com/messages/app/schedule/Scheduling.kt:67
- **What is wrong:** Group recipients are sent sequentially inside one `try`; any later synchronous exception marks every attempt failed, including recipients already handed to the radio.

- **How it fails:** For recipients A, B, C, A's `sendMessage` succeeds and B's throws. `failAllSendAttempts` marks the whole message failed; pressing Resend or the automatic retry sends A again as well as B/C, so A receives a duplicate.
- **Suggested fix:** Dispatch and settle each recipient/part independently, preserve already-dispatched attempts, and retry only attempts that never reached the radio or explicitly failed. Include an attempt generation/idempotency model in UI status.

L-2. Stale SMS callbacks can settle a newer resend

- **Severity:** High
- **Location:**
`core-messaging/src/main/kotlin/com/messages/core/MessageRepository.kt:1206,`
`core-messaging/src/main/kotlin/com/messages/core/MessageRepository.kt:1212,`
`app/src/main/kotlin/com/messages/app/schedule/Scheduling.kt:92,`
`app/src/main/kotlin/com/messages/app/schedule/Scheduling.kt:117`
- **What is wrong:** Resend deletes and recreates attempts with the same deterministic attempt IDs and `PendingIntent` identity; there is no send-generation nonce.
- **How it fails:** Attempt 1 times out and is resent. A delayed success/failure broadcast from attempt 1 arrives after the matrix was recreated and updates attempt 2's row, so the new send can be shown delivered before its callback or failed by an obsolete result.
- **Suggested fix:** Add a monotonically increasing send generation/UUID to the attempt primary key, callback URI, and `PendingIntent` request code; callbacks must update only the matching active generation.

L-3. MMS callback `PendingIntents` collide because their identity is carried only in extras

- **Severity:** High
- **Location:** `app/src/main/kotlin/com/messages/app/receiver/OtherReceivers.kt:77,`
`app/src/main/kotlin/com/messages/app/receiver/OtherReceivers.kt:79,`
`app/src/main/kotlin/com/messages/app/mms/MmsSender.kt:129`
- **What is wrong:** MMS callback intents have no unique action/data URI. `PendingIntent` equality ignores extras, while request codes are a transaction/location hash for download or truncated Long ID for send.
- **How it fails:** Two concurrent download identifiers with the same 32-bit hash, or a resend reusing the same message ID, cause `FLAG_UPDATE_CURRENT` to replace the first token. The first platform callback consumes the second transaction's authority; the other becomes unknown,

losing or mis-setting an MMS.

- **Suggested fix:** Put a cryptographically random transaction token in `Intent.data`, derive a collision-resistant request code as a secondary discriminator, cancel only the exact completed `PendingIntent`, and test concurrent/resend callbacks.

L-4. The received MMS payload is destroyed before durable indexing succeeds

- **Severity:** High
- **Location:** `app/src/main/kotlin/com/messages/app/receiver/OtherReceivers.kt:119`,
`app/src/main/kotlin/com/messages/app/receiver/OtherReceivers.kt:164`,
`app/src/main/kotlin/com/messages/app/receiver/OtherReceivers.kt:191`
- **What is wrong:** The receiver consumes its durable token and deletes the downloaded PDU/revokes grants before `onIncomingMms` has durably stored the provider and Room records.
- **How it fails:** Process death or a repository/storage exception after `settle` but before line 192 destroys the only full PDU. Recovery has no token/file to replay and can create only the generic “couldn't be processed” placeholder, losing the message and attachments.
- **Suggested fix:** Retain the token and PDU in a durable `DOWNLOADED` state until provider + Room indexing commits; `settle/delete` only after success, and retry idempotently after process restart.

L-5. Locked multi-attachment MMS permanently loses all but one attachment

- **Severity:** High
- **Location:**
`core-messaging/src/main/kotlin/com/messages/core/MessageRepository.kt:436`,
`core-messaging/src/main/kotlin/com/messages/core/MessageRepository.kt:473`,
`core-messaging/src/main/kotlin/com/messages/core/MessageRepository.kt:489`
- **What is wrong:** The Room model saves only the first displayable attachment, then locked-space handling purges the Telephony provider copy that contained the complete multipart MMS.
- **How it fails:** Receive a locked MMS containing two photos and an audio clip. Only the first file is copied to app storage; provider purge deletes the PDU/parts, so the other two attachments cannot be recovered or backed up.
- **Suggested fix:** Normalize MMS parts into a one-to-many attachment table, durably copy and verify every supported part before provider purge, and retain/flag the provider row when any copy fails.

L-6. MMS deduplication can drop a legitimate sender or admit a concurrent duplicate

- **Severity:** High
- **Location:**
`core-messaging/src/main/kotlin/com/messages/core/MessageRepository.kt:420,`
`core-messaging/src/main/kotlin/com/messages/core/MessageRepository.kt:429,`
`core-messaging/src/main/kotlin/com/messages/core/MessageRepository.kt:473`
- **What is wrong:** The sole logical key is nullable carrier transaction ID, without sender/carrier context, and its lookup precedes insertion outside a transaction.
- **How it fails:** Sender B reuses sender A's transaction ID and its MMS is discarded as a duplicate; alternatively, two callbacks for one PDU both pass the lookup and insert before either is visible, duplicating the message and notification.
- **Suggested fix:** Use a unique fingerprint over subscription, normalized sender, transaction ID, content-location, and payload digest; insert-or-ignore it atomically and preserve collisions for review instead of silently dropping them.

L-7. SMS rows are inserted into the provider's Sent box before radio dispatch

- **Severity:** High
- **Location:**
`core-messaging/src/main/kotlin/com/messages/core/MessageRepository.kt:930,`
`core-messaging/src/main/kotlin/com/messages/core/MessageRepository.kt:951,`
`core-messaging/src/main/kotlin/com/messages/core/MessageRepository.kt:967`
- **What is wrong:** `storeOutgoing` writes `MESSAGE_TYPE_SENT` provider rows while the app's own row is only `SENDING` and before `SmsManager` accepts the message.
- **How it fails:** The process dies after provider insertion but before `SmsRadio.send`, or radio dispatch throws. Other SMS apps and system consumers show the message as sent even though no transmission occurred; recovery has no definitive way to distinguish it from a real send.
- **Suggested fix:** Insert into Outbox/Queued with a durable send generation, move to Sent only on all sent callbacks, and move to Failed on terminal error. Reconcile provider state from the attempt table.

L-8. An empty parsed recipient list leaves a message pending forever

- **Severity:** Medium

- **Location:** `app/src/main/kotlin/com/messages/app/schedule/Scheduling.kt:41`,
`app/src/main/kotlin/com/messages/app/schedule/Scheduling.kt:45`,
`app/src/main/kotlin/com/messages/app/schedule/Scheduling.kt:51`
- **What is wrong:** Attempt recording coerces recipient count to at least one, but dispatch iterates the original possibly empty list and throws no error.
- **How it fails:** A malformed delimiter-only group address produces zero recipients. One pending attempt is recorded, no SMS is sent and no callback can settle it, so the bubble remains `SENDING` indefinitely.
- **Suggested fix:** Validate a non-empty canonical recipient list before storing/claiming; if empty, atomically mark a local validation failure and tell the user which address is invalid.

L-9. Locked-provider purge and retry treat deleting zero rows as success

- **Severity:** Medium
- **Location:**
`core-messaging/src/main/kotlin/com/messages/core/MessageRepository.kt:605`,
`core-messaging/src/main/kotlin/com/messages/core/MessageRepository.kt:614`,
`core-messaging/src/main/kotlin/com/messages/core/MessageRepository.kt:1866`
- **What is wrong:** Provider deletion success is based on absence of an exception, not on the returned row count, and the retry path clears failure state on a zero-row result.
- **How it fails:** An OEM provider silently returns 0 for a denied/stale URI while the plaintext row still exists. The mapping is treated as deleted/settled, so no future retry removes the locked message's provider copy.
- **Suggested fix:** Require exactly the expected deletion count or verify absence with a query; preserve a retry/tombstone on ambiguity and expose persistent failures in diagnostics/UI.

L-10. Incoming MMS timestamps are download time, not message time

- **Severity:** Medium
- **Location:** `app/src/main/kotlin/com/messages/app/receiver/OtherReceivers.kt:187`,
`app/src/main/kotlin/com/messages/app/receiver/OtherReceivers.kt:192`,
`core-messaging/src/main/kotlin/com/messages/core/mms/MmsPduParser.kt:24`
- **What is wrong:** The parser does not expose the MMS Date header, and intake always passes `System.currentTimeMillis()` after download.
- **How it fails:** A device downloads an MMS hours/days late after being offline. It appears as a new current message, reorders the conversation incorrectly, and backup/duplicate logic records the wrong chronology.

- **Suggested fix:** Parse and validate the PDU date (seconds-to-milliseconds), retain notification/provider dates as fallbacks, and clamp only implausible values with an explicit provenance field.

L-11. Scheduled send cannot promise the user-selected wall-clock time

- **Severity:** Medium
- **Location:** `app/src/main/kotlin/com/messages/app/schedule/Scheduling.kt:252`, `app/src/main/kotlin/com/messages/app/schedule/Scheduling.kt:258`
- **What is wrong:** A plain WorkManager initial delay is used for a user-visible exact scheduled message, with no exact-alarm path or “approximate” disclosure.
- **How it fails:** Under Doze/app standby, a message scheduled for 09:00 can run in a later maintenance window. Time-sensitive greetings, reminders, or business messages are sent materially late while the UI represented a precise time.
- **Suggested fix:** Either label scheduling as approximate and show the actual constraint, or use the platform exact-alarm mechanism with the required permission/policy flow and a WorkManager fallback; warn when exact scheduling is unavailable.

L-12. Multi-SIM selection is unreachable because its permission is never requested

- **Severity:** Medium
- **Location:** `app/src/main/AndroidManifest.xml:22`, `app/src/main/kotlin/com/messages/app/MainActivity.kt:806`, `app/src/main/kotlin/com/messages/app/sms/SimChoices.kt:30`
- **What is wrong:** `READ_PHONE_STATE` is declared and required by `SimChoices`, but `requestCorePermissions` omits it.
- **How it fails:** On a fresh dual-SIM install the check always returns empty, the SIM picker is hidden, and sends use the platform default even when the user expected the other subscription.
- **Suggested fix:** Request the permission contextually when the user opens SIM selection, explain why, and provide a system subscription-picker/fallback that does not silently pretend only one route exists.

L-13. MMS registry persistence failure is ignored before platform handoff

- **Severity:** Medium

- **Location:** `app/src/main/kotlin/com/messages/app/mms/MmsTransactions.kt:78`,
`app/src/main/kotlin/com/messages/app/mms/MmsTransactions.kt:97`,
`app/src/main/kotlin/com/messages/app/receiver/OtherReceivers.kt:72`
- **What is wrong:** `register` returns a token even if synchronous `SharedPreferences.commit()` fails.
- **How it fails:** With storage full, the app hands the platform a callback token and URI, but the callback cannot resolve a record. A received MMS becomes an unknown-token placeholder; a sent MMS never updates its bubble correctly and its grant/temp PDU can linger.
- **Suggested fix:** Throw/return failure unless the record is durably verified, and do not invoke `SmsManager` until registration succeeds. Prefer a transactional Room registry.

L-14. MMS send startup exceptions leak the PDU and URI grants

- **Severity:** Medium
- **Location:** `app/src/main/kotlin/com/messages/app/mms/MmsSender.kt:115`,
`app/src/main/kotlin/com/messages/app/mms/MmsSender.kt:126`,
`app/src/main/kotlin/com/messages/app/mms/MmsSender.kt:139`,
`app/src/main/kotlin/com/messages/app/mms/MmsSender.kt:145`
- **What is wrong:** After creating/registering/granting the PDU, the catch path only marks the message failed; it does not settle the registry record, revoke grants, or delete the file.
- **How it fails:** `sendMultimediaMessage` throws synchronously due to an invalid subscription or carrier service failure. The cache PDU and telephony read grants remain until a later age-based sweep, exposing content longer and consuming disk.
- **Suggested fix:** Track the record locally and call idempotent `settle` in every pre-dispatch failure path; distinguish accepted-by-platform from not-started state.

L-15. Carrier spam reports have no part/cost bound and claim success too early

- **Severity:** Medium
- **Location:** `app/src/main/kotlin/com/messages/app/report/CarrierReport.kt:20`,
`app/src/main/kotlin/com/messages/app/report/CarrierReport.kt:25`
- **What is wrong:** Arbitrary report text is split into unlimited SMS parts, sent without sent/delivery callbacks, and returns `true` merely when the synchronous call did not throw.
- **How it fails:** A very long reported body generates hundreds of chargeable short-code parts; the UI reports success even if the carrier later rejects all of them.

- **Suggested fix:** Enforce a documented byte/part ceiling, preview possible charges, require confirmation, and track sent callbacks per part before presenting success.

L-16. The main composer has no message-length or part-cost ceiling

- **Severity:** Medium
- **Location:** `app/src/main/kotlin/com/messages/app/ui/chat/ChatViewModel.kt:118`, `app/src/main/kotlin/com/messages/app/receiver/OtherReceivers.kt:368`
- **What is wrong:** Inline reply has a bounded input path, but the full composer accepts an arbitrarily long string and sends every `divideMessage` part without a cap or cost confirmation.
- **How it fails:** Pasting a megabyte of text creates thousands of SMS segments/attempt rows and can incur extreme carrier charges or binder/radio failures.
- **Suggested fix:** Show live segment/encoding count, enforce a conservative maximum, require explicit confirmation above a small multipart threshold, and offer MMS/file sharing for large content.

L-17. Group-recipient selection lacks a cap and canonical uniqueness

- **Severity:** Medium
- **Location:** `app/src/main/kotlin/com/messages/app/ui/compose/NewMessageScreen.kt:180`, `app/src/main/kotlin/com/messages/app/ui/compose/NewMessageScreen.kt:183`, `app/src/main/kotlin/com/messages/app/ui/compose/NewMessageScreen.kt:248`
- **What is wrong:** Recipient uniqueness strips punctuation but does not canonicalize country/national equivalents, and the selected-recipient list has no upper bound.
- **How it fails:** Select `+81 90...` and `090...` for the same person; both survive and receive duplicate SMS. Selecting hundreds of contacts creates an enormous fan-out/attempt matrix and cost exposure.
- **Suggested fix:** Canonicalize with libphonenumber plus subscription/region context, cap recipients and estimated parts/cost, and require confirmation for bulk sends.

L-18. Short/service-code-like free text is accepted as a normal recipient

- **Severity:** Medium
- **Location:** `app/src/main/kotlin/com/messages/app/ui/compose/NewMessageScreen.kt:198`, `app/src/main/kotlin/com/messages/app/ui/compose/NewMessageScreen.kt:325`

- **What is wrong:** The manual-address predicate treats arbitrary three-digit numeric strings as dialable ordinary recipients.
- **How it fails:** A mistyped value such as 123 is accepted and can route to a premium/service short code instead of failing validation, potentially creating an unexpected charge or command.
- **Suggested fix:** Treat short codes as a separate high-risk recipient type, validate with carrier/region rules where available, and show a prominent confirmation including possible charges.

Backup & Restore Issues

R-1. Failed restore rollback leaves damaged Room rows that block a clean retry

- **Severity:** High
- **Location:**
`core-messaging/src/main/kotlin/com/messages/core/backup/BackupManager.kt:722,`
`core-messaging/src/main/kotlin/com/messages/core/backup/BackupManager.kt:848,`
`core-messaging/src/main/kotlin/com/messages/core/backup/BackupManager.kt:894`
- **What is wrong:** Rollback deletes provider rows/media and restores settings, but deliberately retains every Room message/rule/reputation inserted before the exception.
- **How it fails:** A restore inserts 500 Room rows and provider copies, then fails. Rollback removes the provider/media side effects but leaves the Room rows pointing at now-deleted provider/media data. Retrying treats them as duplicates, so the missing provider copies/files are never recreated and the restore can never converge to the intended snapshot.
- **Suggested fix:** Stage and validate into temporary tables/files, then commit Room in one transaction after external writes succeed, or journal every inserted Room primary key and delete/reconcile exactly those on rollback. A retry must repair incomplete side effects rather than skip by content alone.

R-2. Drive key and checkpoint state cross Google-account boundaries

- **Severity:** High
- **Location:** `app/src/main/kotlin/com/messages/app/drive/DriveBackup.kt:70,`
`app/src/main/kotlin/com/messages/app/drive/DriveBackup.kt:181,`
`app/src/main/kotlin/com/messages/app/drive/DriveBackup.kt:208,`
`app/src/main/kotlin/com/messages/app/ui/drivebackup/DriveBackupScreen.kt:571`
- **What is wrong:** Master-key cache, custody, last-covered checkpoint, and status preferences are global, not keyed by signed-in account; sign-out clears none of them.

- **How it fails:** Back up account A, sign out, then sign into empty account B. B can inherit A's `last_checkpoint_covered` and skip its first automatic backup, or `ensureMasterKey` uploads A's cached key into B, coupling two accounts' backup confidentiality and recovery state.
- **Suggested fix:** Namespace every Drive preference, work name, and Keystore alias by a stable hashed Google account ID. On account change cancel work, clear in-memory/cache state, and require explicit migration/confirmation before copying a key between accounts.

R-3. Backup decryption allocates the entire compressed body before applying expansion bounds

- **Severity:** High
- **Location:**
`core-messaging/src/main/kotlin/com/messages/core/backup/BackupCrypto.kt:64,`
`core-messaging/src/main/kotlin/com/messages/core/backup/BackupCrypto.kt:358,`
`core-messaging/src/main/kotlin/com/messages/core/backup/BackupCrypto.kt:371`
- **What is wrong:** A blob may be up to 512 MiB; `Cipher.doFinal` creates a full plaintext-compressed array before bounded decompression begins, then decompression and `String` add more copies.
- **How it fails:** Select a validly encrypted near-512-MiB restore file on a typical phone. Authentication/decryption allocates hundreds of MiB at once and the process is killed/OOMs before `gunzipBounded` can enforce the device-derived expanded limit.
- **Suggested fix:** Lower the envelope cap to a realistic device budget and stream authenticated chunks using a chunked AEAD format with per-chunk nonces/tags and a signed manifest; avoid materializing blob, decrypted compressed bytes, expanded bytes, and JSON string simultaneously.

R-4. Normal-media export has no aggregate byte or file-count limit

- **Severity:** High
- **Location:**
`core-messaging/src/main/kotlin/com/messages/core/backup/BackupManager.kt:198,`
`core-messaging/src/main/kotlin/com/messages/core/backup/BackupManager.kt:202,`
`core-messaging/src/main/kotlin/com/messages/core/backup/BackupManager.kt:206`
- **What is wrong:** Each normal attachment is individually bounded, but unlike locked media there is no total bytes/files budget before bytes are Base64-encoded into a map and then serialized.
- **How it fails:** A mailbox with hundreds of near-limit attachments causes all byte arrays, 4/3-size Base64 strings, JSON, gzip output, and encrypted output to coexist, exhausting heap and

potentially disk; scheduled backup repeatedly fails.

- **Suggested fix:** Apply the same aggregate `MAX_MEDIA_BYTES/MAX_MEDIA_FILES` budget before reads, stream media as separate encrypted chunks, and report exactly which files were excluded.

R-5. A successful snapshot can silently omit every locked chat

- **Severity:** High

- **Location:**

```
core-messaging/src/main/kotlin/com/messages/core/backup/BackupManager.kt:282,  
core-messaging/src/main/kotlin/com/messages/core/backup/BackupManager.kt:294,  
core-messaging/src/main/kotlin/com/messages/core/backup/BackupManager.kt:296
```

- **What is wrong:** If locked rows exist but the locally cached KEK/salt is unavailable, `lockedEnvelope` returns `null` (unless a prior pending envelope exists) and outer backup proceeds successfully.
- **How it fails:** Keystore cache loss or a fresh process without a usable KEK occurs before the scheduled backup. The snapshot reports success but contains no locked conversations; if the device is then lost/reset, those chats cannot be recovered from Drive.
- **Suggested fix:** Treat locked-row omission as a blocking backup failure requiring unlock, or produce an explicit partial snapshot state that cannot advance/prune the last complete checkpoint. Show locked/normal counts before and after upload.

R-6. Content-key loss can back up device-bound ciphertext as if it were message text

- **Severity:** High

- **Location:**

```
core-messaging/src/main/kotlin/com/messages/core/backup/BackupManager.kt:282,  
core-messaging/src/main/kotlin/com/messages/core/backup/BackupManager.kt:290,  
core-messaging/src/main/kotlin/com/messages/core/secret/LockedContent.kt:139
```

- **What is wrong:** `LockedContent.open` returns an undecryptable marker-prefixed ciphertext unchanged; backup then serializes that value inside the portable locked envelope without checking that opening succeeded.
- **How it fails:** Lose/invalidate the content key while the credential KEK is still available, then back up. Restore on another device writes the opaque old ciphertext; its marker makes sealing/opening treat it as already sealed, but the old device-only key no longer exists, so the message is permanently unreadable.

- **Suggested fix:** Make opening return a typed success/failure, fail the snapshot if any locked row cannot decrypt, and include authenticated per-row plaintext/ciphertext-state validation in the sub-envelope.

R-7. A second restore overwrites the first pending locked envelope

- **Severity:** High
- **Location:**
`core-messaging/src/main/kotlin/com/messages/core/backup/BackupManager.kt:822,`
`core-messaging/src/main/kotlin/com/messages/core/backup/BackupManager.kt:838,`
`core-messaging/src/main/kotlin/com/messages/core/secret/SecretSpace.kt:313`
- **What is wrong:** Only one fixed pending blob/auth slot exists and `storePendingRestore` replaces it without merge, identity check, or warning.
- **How it fails:** Restore backup A but defer entering its old PIN; then restore backup B. B overwrites A's opaque envelope, so A's locked chats are irretrievable even though the first restore reported them as pending.
- **Suggested fix:** Store pending envelopes by authenticated snapshot ID, list them to the user, and require unlock/explicit discard before replacing. Merge only after each envelope authenticates and imports idempotently.

R-8. A hostile header can force sixteen million PBKDF2 iterations per attempt

- **Severity:** High
- **Location:**
`core-messaging/src/main/kotlin/com/messages/core/backup/BackupCrypto.kt:64,`
`core-messaging/src/main/kotlin/com/messages/core/backup/BackupCrypto.kt:67,`
`core-messaging/src/main/kotlin/com/messages/core/backup/BackupCrypto.kt:203,`
`core-messaging/src/main/kotlin/com/messages/core/backup/BackupCrypto.kt:313`
- **What is wrong:** Validation permits eight password wrappers, each with two million iterations, and a wrong password tries every wrapper sequentially.
- **How it fails:** An attacker supplies a structurally valid restore header containing eight maximum-cost wraps. Every password attempt performs 16,000,000 PBKDF2 rounds before rejecting it, tying up a worker/CPU and draining battery; repeated UI attempts compound the cost.
- **Suggested fix:** Limit password wrappers to the actual supported use case (normally one), enforce an aggregate KDF-work budget, rate-limit attempts, and perform restore in a cancellable foreground job with explicit progress.

R-9. Invalid locked envelopes can be silently counted as a successful outer restore

- **Severity:** Medium
- **Location:**
`core-messaging/src/main/kotlin/com/messages/core/backup/BackupManager.kt:822,`
`core-messaging/src/main/kotlin/com/messages/core/backup/BackupManager.kt:826,`
`core-messaging/src/main/kotlin/com/messages/core/backup/BackupManager.kt:838`
- **What is wrong:** Base64, header, unwrap, or authentication errors are collapsed to null; without parseable `LockedAuth`, the code neither imports nor stores a pending envelope nor fails the restore.
- **How it fails:** Corrupt one byte in `LockedEnvelope` or its auth metadata. Normal messages restore and the operation returns stats, while all locked content is silently skipped with no actionable error.
- **Suggested fix:** Distinguish absent, credential-required, corrupt, unsupported, and authenticated states; fail or explicitly mark the restore partial, preserve the original blob for recovery, and never present full success.

R-10. The snapshot schema loses MMS, group, SIM, and delivery-state fidelity

- **Severity:** Medium
- **Location:**
`core-messaging/src/main/kotlin/com/messages/core/backup/BackupManager.kt:52,`
`core-messaging/src/main/kotlin/com/messages/core/backup/BackupManager.kt:729,`
`core-messaging/src/main/kotlin/com/messages/core/backup/BackupManager.kt:796`
- **What is wrong:** `BackupMessage` lacks message kind, MMS transaction/parts, group membership, subscription ID, send attempt/result/delivery data, and original provider state; restore maps every non-trashed item to one SMS Inbox/Sent row and every outgoing row to `SENT`.
- **How it fails:** Restore a sent MMS/group message from SIM 2 with a failed delivery part. It returns as an SMS-like row, loses all but the one optional media blob and SIM, and is falsely shown as fully sent.
- **Suggested fix:** Version a lossless message envelope with explicit transport, recipients, parts, subscription identity, status provenance, and provider representation. Migrate/validate by version and disclose fields unavailable on the destination device.

R-11. Missing or unreadable media is omitted without making the snapshot partial

- **Severity:** Medium

- **Location:**

core-messaging/src/main/kotlin/com/messages/core/backup/BackupManager.kt:198,
core-messaging/src/main/kotlin/com/messages/core/backup/BackupManager.kt:252,
core-messaging/src/main/kotlin/com/messages/core/backup/BackupManager.kt:753

- **What is wrong:** Export's `readMedia(...)? : return@forEach` silently drops missing, oversized, denied, or unreadable files; restore also swallows media decode/write errors.
- **How it fails:** Revoke a provider URI or remove one local attachment, create an “include media” backup, then restore it. The message is present with no attachment and there was no warning or manifest evidence that the snapshot was incomplete.
- **Suggested fix:** Include an authenticated attachment manifest with expected digest/size/status, fail full-backup mode on omission or require explicit partial acceptance, and return detailed export/import stats.

R-12. Duplicate rules inside one backup all insert

- **Severity:** Medium

- **Location:**

core-messaging/src/main/kotlin/com/messages/core/backup/BackupManager.kt:682,
core-messaging/src/main/kotlin/com/messages/core/backup/BackupManager.kt:685

- **What is wrong:** Duplicate detection compares each incoming rule only against the pre-restore `existingRules` snapshot, never adding newly inserted rules to that set.
- **How it fails:** A crafted/legacy backup contains the same custom regex ten times. With no preexisting copy, all ten insert and execute on every message, multiplying work and duplicating matches.
- **Suggested fix:** Deduplicate the validated incoming list first and enforce a database unique index over canonical `(kind, target, pattern)`; use insert-or-ignore and count conflicts.

R-13. Pattern-pack import failure is discarded

- **Severity:** Medium

- **Location:**

core-messaging/src/main/kotlin/com/messages/core/backup/BackupManager.kt:678,
core-messaging/src/main/kotlin/com/messages/core/MessageRepository.kt:143

- **What is wrong:** `importPatternPack` returns a `Result`, but restore ignores it and proceeds as successful.

- **How it fails:** Restore a snapshot with an invalid/signature-incompatible pattern pack. Messages/settings import, the protection pack does not, and the user is told the restore succeeded while classifier behavior differs from the snapshot.
- **Suggested fix:** Inspect the `Result`; either reject before mutation during validation or record a clearly surfaced partial failure with no checkpoint advancement.

R-14. Legitimate identical messages at one timestamp are collapsed

- **Severity:** Medium
- **Location:**
`core-messaging/src/main/kotlin/com/messages/core/backup/BackupManager.kt:1058,`
`core-messaging/src/main/kotlin/com/messages/core/backup/BackupManager.kt:1088`
- **What is wrong:** Restore identity is only address, timestamp, direction, and body digest; it has no source/provider ID or occurrence ordinal.
- **How it fails:** Two genuinely distinct carrier messages with the same body and coarse identical timestamp from the same sender are in one backup. The second is counted as a duplicate and permanently omitted.
- **Suggested fix:** Export a stable message UUID/source ID plus occurrence sequence. For legacy backups, retain multiplicity within a snapshot and use a richer fuzzy match only against existing device data.

R-15. Existing stale category always overrides a newer restored message

- **Severity:** Medium
- **Location:**
`core-messaging/src/main/kotlin/com/messages/core/backup/BackupManager.kt:1117,`
`core-messaging/src/main/kotlin/com/messages/core/backup/BackupManager.kt:1127,`
`core-messaging/src/main/kotlin/com/messages/core/backup/BackupManager.kt:1130`
- **What is wrong:** Even when `latest.timestamp` wins, rebuilt conversation category is `existing?.category ?: latest.category`, so any existing category is retained.
- **How it fails:** A stale Inbox conversation exists, then restore adds a newer Spam/Fraud message. Preview/timestamp update but the conversation remains in Inbox, defeating the restored classification and folder placement.
- **Suggested fix:** Derive category from the latest live message unless there is a separately modeled user override; persist override provenance instead of overloading the category field.

R-16. Drive error responses are read without a bound

- **Severity:** Medium
- **Location:** `app/src/main/kotlin/com/messages/app/drive/DriveClient.kt:221`,
`app/src/main/kotlin/com/messages/app/drive/DriveClient.kt:223`
- **What is wrong:** `Non-2xx errorStream.readBytes()` is unbounded even though only the first 400 characters are logged/thrown.
- **How it fails:** A compromised system-trusted proxy or anomalous endpoint returns a very large error body; the app allocates it in full and can OOM during backup/restore.
- **Suggested fix:** Read at most a small fixed number of bytes through the bounded reader, close immediately, and preserve only a redacted status/error code.

R-17. A capped Drive download does not prove end-of-stream

- **Severity:** Medium
- **Location:** `app/src/main/kotlin/com/messages/app/drive/DriveClient.kt:249`,
`app/src/main/kotlin/com/messages/app/drive/DriveClient.kt:256`,
`app/src/main/kotlin/com/messages/app/drive/DriveClient.kt:262`
- **What is wrong:** When exactly `maxBytes` have been read, the loop stops without probing one extra byte, so an oversized chunked response is returned as if complete.
- **How it fails:** Drive/proxy sends `maxBytes + 1` bytes with unknown length. The client returns a truncated blob; crypto later reports corruption/wrong key instead of the accurate size violation, frustrating recovery and retry decisions.
- **Suggested fix:** Read through a `maxBytes + 1` sentinel and throw a dedicated size exception if the extra byte exists; do not return truncated data.

R-18. Generated recovery codes are committed without proof the user saved them

- **Severity:** Medium
- **Location:**
`app/src/main/kotlin/com/messages/app/ui/drivebackup/DriveBackupScreen.kt:338`,
`app/src/main/kotlin/com/messages/app/ui/drivebackup/DriveBackupScreen.kt:351`,
`core-messaging/src/main/kotlin/com/messages/core/backup/MasterKeyVault.kt:127`
- **What is wrong:** The generated-code step proceeds directly with the displayed value; there is no re-entry/word challenge or verified export.

- **How it fails:** A user taps through without saving the code, later loses the device/local Keystore cache, and the user-held Drive vault becomes permanently unrecoverable.
- **Suggested fix:** Require re-entry of randomly selected groups or verified save to a user-chosen file before deleting the plaintext account key, with an explicit cancellation rollback.

R-19. Auth-recovery cancellation is treated as success and blindly retried

- **Severity:** Medium
- **Location:**
`app/src/main/kotlin/com/messages/app/ui/drivebackup/DriveBackupScreen.kt:142,`
`app/src/main/kotlin/com/messages/app/ui/drivebackup/DriveBackupScreen.kt:520`
- **What is wrong:** The activity-result callback ignores result code/data and always invokes the stashed retry.
- **How it fails:** The user cancels Google's recovery/consent screen; the same Drive call is immediately retried, can prompt again or fail confusingly, while pending retry state has already been cleared.
- **Suggested fix:** Inspect `Activity.RESULT_OK`, retain/cancel retry state explicitly, surface cancellation, and prevent prompt loops with a bounded recovery state machine.

R-20. Snapshot message count does not describe snapshot contents

- **Severity:** Low
- **Location:** `app/src/main/kotlin/com/messages/app/drive/DriveBackup.kt:637,`
`app/src/main/kotlin/com/messages/app/drive/DriveBackup.kt:652`
- **What is wrong:** Header/status count includes all non-trash, non-scheduled rows before spam shaping and without knowing whether locked content/media was omitted.
- **How it fails:** Choose “exclude spam” or lose the locked KEK; Drive UI/header can claim hundreds more messages than the serialized payload actually contains, undermining backup verification.
- **Suggested fix:** Compute authenticated counts from the finalized payload, broken down by normal/locked/spam/media and omitted/error reason, and verify those counts during restore.

Database Schema & Migration Issues

DB-1. Upgraded databases never receive Room's FTS synchronization triggers

- **Severity:** High

- **Location:** `core-messaging/src/main/kotlin/com/messages/core/db/Migrations.kt:122`,
`core-messaging/src/main/kotlin/com/messages/core/db/Migrations.kt:126`,
`core-messaging/schemas/com.messages.core.db.MessagesDatabase/10.json:505`
- **What is wrong:** Migration 5→6 creates/rebuilds the external-content FTS4 table but does not create the four insert/update/delete triggers present in Room's v10 schema.
- **How it fails:** Upgrade a real v5 database, then receive, edit/recode, or delete messages. `messages_fts` is not kept in sync: new messages are absent from search and deleted/locked plaintext can remain searchable in the stale FTS index.
- **Suggested fix:** Add an idempotent migration that creates the exact Room-generated triggers and rebuilds FTS; bump the schema version and add migration tests starting from every committed schema, asserting insert/update/delete search behavior and locked-content removal.

Classifier, Spam & Fraud Protection Issues

F-1. Any OTP match overrides even explicit malware and phishing evidence

- **Severity:** High
- **Location:**
`protection-engine/src/main/kotlin/com/messages/protection/ProtectionEngine.kt:99`,
`protection-engine/src/main/kotlin/com/messages/protection/ProtectionEngine.kt:115`,
`protection-engine/src/main/kotlin/com/messages/protection/ProtectionEngine.kt:260`
- **What is wrong:** OTP protection is absolute; a syntactically “registered” sender also suppresses the protected-verdict phishing warning.
- **How it fails:** Classify sender `AX-BANKXX` with `OTP 123456; install https://evil.xyz/payload.apk`. OTP makes the engine return from the protected lane before C9/scam scoring, and sender syntax makes `warn=false`; the malware link lands in Inbox with no fraud banner.
- **Suggested fix:** Never let protected content erase independent high-confidence evidence such as APK, credential request, IP/punycode, or known-phishy link. Deliver the OTP if desired, but retain a dangerous banner/quarantine state and show both reasons.

F-2. “Verified” status is inferred from an unverified sender string

- **Severity:** High
- **Location:**
`protection-engine/src/main/kotlin/com/messages/protection/SenderAnalyzer.kt:11`,
`protection-engine/src/main/kotlin/com/messages/protection/SenderAnalyzer.kt:27`,

protection-engine/src/main/kotlin/com/messages/protection/SenderBadges.kt:35,
protection-engine/src/main/kotlin/com/messages/protection/SenderBadges.kt:69

- **What is wrong:** Matching the DLT-shaped regex alone classifies a sender as registered and displays “Registered business sender (DLT header),” with no registry lookup, signature, carrier attestation, or known-domain binding.
- **How it fails:** A spoofed/gateway sender presented as AX-BANKXX gets trust multipliers, protected-lane eligibility, a VERIFIED badge, and suppressed link warnings even though the app has established only string shape.
- **Suggested fix:** Rename the state to neutral “business-style sender ID” unless verified against trustworthy carrier/registry data. Do not grant security exemptions from syntax; bind a verified entity to expected domains and revoke trust on conflicting evidence.

F-3. Regex protection has a per-pattern budget but no per-message budget

- **Severity:** High
- **Location:**
protection-engine/src/main/kotlin/com/messages/protection/SafeRegexPolicy.kt:130,
protection-engine/src/main/kotlin/com/messages/protection/SafeRegexPolicy.kt:261,
protection-engine/src/main/kotlin/com/messages/protection/PatternPackPolicy.kt:21,
protection-engine/src/main/kotlin/com/messages/protection/PatternMatcher.kt:25
- **What is wrong:** Static validation misses ambiguous quantified alternatives such as (a|aa)+\$, and each of up to 500 patterns receives a fresh two-million-character-read budget.
- **How it fails:** Import 500 accepted ambiguous patterns and classify a long aaaa...X body. Every match backtracks until its individual budget and fails open, permitting roughly one billion indexed reads on one intake, stalling classification and draining CPU even though no single regex runs forever.
- **Suggested fix:** Reject ambiguous-overlap constructs with a real regex AST/analyzer or use a linear-time engine; share one cancellable deadline/read budget across the entire message and disable/report patterns that exhaust it.

F-4. Multi-megabyte MMS text reaches an uncapped copy-heavy normalizer

- **Severity:** High
- **Location:**
core-messaging/src/main/kotlin/com/messages/core/mms/MmsPduParser.kt:93,
protection-engine/src/main/kotlin/com/messages/protection/Normalizer.kt:56,
protection-engine/src/main/kotlin/com/messages/protection/Normalizer.kt:62,

protection-engine/src/main/kotlin/com/messages/protection/Normalizer.kt:65

- **What is wrong:** The MMS boundary allows an 8 MiB PDU, while normalization has no body-size ceiling and repeatedly creates NFKC, zero-width, homoglyph, entity-list, de-leet, lowercase, and whitespace-replacement copies.
- **How it fails:** A carrier-controlled MMS with a multi-megabyte text part causes multiple large strings/lists plus hundreds of pattern scans, producing long intake latency or OOM before the message can be safely indexed/notified.
- **Suggested fix:** Impose a classifier input ceiling, classify a bounded prefix plus structured link/entity samples, record truncation in the verdict, and stream/limit extraction. Preserve the full body separately for display only if storage budgets allow.

F-5. Sender risk is hard-coded for India on every device

- **Severity:** Medium

- **Location:**

protection-engine/src/main/kotlin/com/messages/protection/SenderAnalyzer.kt:15,
protection-engine/src/main/kotlin/com/messages/protection/SenderAnalyzer.kt:23,
core-messaging/src/main/kotlin/com/messages/core/MessageRepository.kt:708

- **What is wrong:** The production caller never supplies locale/SIM country, so home country remains +91 and only Indian mobile formats receive personal-number treatment.
- **How it fails:** On a Japanese device, a normal +81... sender is labeled international and scam-family weights double; legitimate money/job wording is more likely spam. National-format Japanese numbers fall into alphanumeric-unknown logic.
- **Suggested fix:** Derive region per subscription/network/user setting, normalize with libphonenumber, and use locale-specific sender models/pattern packs with an explicit unknown-region fallback.

F-6. Protective “do not share OTP” wording is itself labeled fraud

- **Severity:** Medium

- **Location:**

protection-engine/src/main/kotlin/com/messages/protection/ComboRules.kt:21,
protection-engine/src/main/kotlin/com/messages/protection/ComboRules.kt:75

- **What is wrong:** C7 matches the verbs share/send/give/provide near credential words without recognizing negation or safety-advice context, then marks the result “always fraud.”
- **How it fails:** A legitimate message such as For your safety, do not share your OTP or PIN with anyone triggers C7 and can be marked dangerous/spam when no numeric OTP

protected pattern overrides it.

- **Suggested fix:** Add scoped negation/safety-phrase handling, require an imperative directed at the recipient or a destination/callback signal, and include these exact advisories as negative regression cases across supported languages.

F-7. Fraud combinations override the saved-contact trust path too broadly

- **Severity:** Medium
- **Location:**
`protection-engine/src/main/kotlin/com/messages/protection/ComboRules.kt:52,`
`protection-engine/src/main/kotlin/com/messages/protection/ProtectionEngine.kt:126`
- **What is wrong:** C2 ignores sender context and is computed before the contact bypass; any prize word plus amount is dangerous even in ordinary conversation.
- **How it fails:** A saved contact texts `I won $500 at the office raffle`. C2 enters `fraudCombos`, prevents the saved-contact Inbox return, and sends the personal message to Spam with a fraud warning.
- **Suggested fix:** Require solicitation/claim instructions or unknown-sender/link evidence for C2; apply contact context to combo design rather than as an all-or-nothing late bypass.

F-8. APK malware detection is case- and suffix-fragile

- **Severity:** Medium
- **Location:**
`protection-engine/src/main/kotlin/com/messages/protection/LinkAnalyzer.kt:114`
- **What is wrong:** The regex recognizes lowercase `.apk` only when followed by `?` or end-of-string.
- **How it fails:** `https://evil.example/Payload.APK` and `https://evil.example/payload.apk#download` avoid the APK signal/C9, reducing a direct malware message's score and potentially leaving it unflagged.
- **Suggested fix:** Parse the URI path, percent-decode safely, compare the terminal extension case-insensitively, and ignore query/fragment when determining file type.

F-9. Suspicious scheme-less domains are listed but never extracted

- **Severity:** Medium
- **Location:**
`protection-engine/src/main/kotlin/com/messages/protection/Normalizer.kt:42,`

protection-engine/src/main/kotlin/com/messages/protection/LinkAnalyzer.kt:32

- **What is wrong:** The suspicious set includes `.zip`, `.mov`, `.bond`, `.fun`, `.host`, `.space`, and `.website`, but the scheme-less URL regex omits them.
- **How it fails:** A text containing `claim-now.zip` without `https://` produces no URL entity, so suspicious-TLD, combo, and brand checks never run.
- **Suggested fix:** Use a maintained public-suffix/URI extractor with a bounded input policy, or generate both extraction and risk lists from the same canonical TLD data and test every listed suffix.

F-10. Brand impersonation uses label substrings instead of DNS labels

- **Severity:** Medium
- **Location:**
protection-engine/src/main/kotlin/com/messages/protection/LinkAnalyzer.kt:42,
protection-engine/src/main/kotlin/com/messages/protection/LinkAnalyzer.kt:88
- **What is wrong:** `domain.contains(brand)` treats any occurrence as an impersonated brand.
- **How it fails:** A legitimate link to `amazonaws.com` contains `amazon` but is not in the official list, so it receives a high-risk brand-impersonation signal and dangerous C6 verdict.
- **Suggested fix:** Compare normalized registrable domains and labels, maintain explicit brand-domain mappings, and avoid inferring a brand from arbitrary substrings.

F-11. Locale-default lowercasing makes classification device-locale dependent

- **Severity:** Medium
- **Location:**
protection-engine/src/main/kotlin/com/messages/protection/Normalizer.kt:76
- **What is wrong:** `lowercase()` uses the current default locale rather than locale-invariant case folding.
- **How it fails:** Under Turkish locale, Latin `I` becomes dotless `ı`; English keywords/domains containing `I` can stop matching the bundled case assumptions, creating device-specific false negatives.
- **Suggested fix:** Use `lowercase(Locale.ROOT)`/Unicode case folding for machine matching and add Turkish/Azeri locale regression tests.

F-12. Pattern language metadata is never enforced or selected

- **Severity:** Low
- **Location:** `protection-engine/src/main/kotlin/com/messages/protection/Model.kt:68`,
`protection-engine/src/main/kotlin/com/messages/protection/PatternMatcher.kt:21`
- **What is wrong:** Each pattern declares languages, but `PatternMatcher` compiles and runs every pattern against every message without detecting script/language.
- **How it fails:** As packs grow, language-specific homographs/keywords produce avoidable false positives in other languages and every message pays the cost of irrelevant patterns; pack authors may wrongly believe metadata controls matching.
- **Suggested fix:** Either remove the misleading field or implement script/language routing with a conservative unknown-language lane and cross-language safety tests.

Error Handling Issues

E-1. Classification fallback catches cancellation and fatal VM errors

- **Severity:** High
- **Location:**
`core-messaging/src/main/kotlin/com/messages/core/MessageRepository.kt:742`,
`core-messaging/src/main/kotlin/com/messages/core/MessageRepository.kt:747`
- **What is wrong:** `runCatching` catches every `Throwable`, including `CancellationException`, `OutOfMemoryError`, and other fatal errors, then tries to log/allocate an Inbox verdict.
- **How it fails:** Receiver timeout cancels classification, but the cancellation is converted to a normal Inbox result and processing continues after its lifecycle budget. Under OOM the handler attempts more allocation/logging instead of allowing controlled process recovery, potentially leaving partial intake state.
- **Suggested fix:** Rethrow `CancellationException` and fatal `Error` types; catch only anticipated classifier initialization/match exceptions. Put the safe Inbox fallback at a durable intake boundary with explicit error telemetry.

E-2. Four screens render loading/query failure as a genuine empty mailbox

- **Severity:** Medium
- **Location:** `app/src/main/kotlin/com/messages/app/ui/archived/ArchivedScreen.kt:59`,
`app/src/main/kotlin/com/messages/app/ui/archived/ArchivedScreen.kt:102`,
`app/src/main/kotlin/com/messages/app/ui/starred/StarredScreen.kt:63`,
`app/src/main/kotlin/com/messages/app/ui/outbox/OutboxScreen.kt:103`,
`app/src/main/kotlin/com/messages/app/ui/trash/TrashScreen.kt:71`

- **What is wrong:** Archived, Starred, Outbox, and Trash use `emptyList()` as `stateIn`'s loading seed and have no typed loading/error state; their screens equate empty with successful query completion.
- **How it fails:** On entry, users briefly see “empty”; if the Room/contact mapping flow throws, collection terminates at empty forever, falsely claiming there are no archived/starred/scheduled/trashed messages and offering no retry.
- **Suggested fix:** Expose `Loading | Ready(list) | Failed(error)` state, preserve the last good list, render skeleton/retry/error separately, and restart terminated flows on retry.

E-3. Secret setup, import, and reset exceptions strand permanent busy states

- **Severity:** Medium
- **Location:**
`app/src/main/kotlin/com/messages/app/ui/secret/SecretSetupScreen.kt:77,`
`app/src/main/kotlin/com/messages/app/ui/secret/SecretPromptScreen.kt:94,`
`app/src/main/kotlin/com/messages/app/ui/secret/SecretPromptScreen.kt:289`
- **What is wrong:** Coroutines set `working`, `checking/importing`, or `resetting` before unguarded operations and clear them only on the success path.
- **How it fails:** Keystore, migration, pending-restore, provider delete, or storage failure throws. The coroutine ends, controls remain disabled/spinning for the composition lifetime, and no error explains whether data/setup partially changed.
- **Suggested fix:** Use `try/catch/finally`, a typed operation state, and transactional/idempotent domain operations; clear secrets and busy flags in `finally`, surface retryable vs terminal failures, and verify partial-state recovery.

E-4. Failed or cancelled backfill is displayed as “running” forever

- **Severity:** Medium
- **Location:**
`app/src/main/kotlin/com/messages/app/ui/onboarding/OnboardingScreen.kt:190,`
`app/src/main/kotlin/com/messages/app/ui/onboarding/OnboardingScreen.kt:194,`
`app/src/main/kotlin/com/messages/app/ui/onboarding/OnboardingScreen.kt:210`
- **What is wrong:** Only `SUCCEEDED` is terminal; `FAILED`, `CANCELLED`, and `BLOCKED` all fall through to the `progress/running` branch.
- **How it fails:** Backfill fails once due to database/storage error. Onboarding shows an indefinite spinner and “running/scanning,” with no error, retry, or diagnostic path.

- **Suggested fix:** Exhaustively map every `workInfo.State`, show failure/cancellation with retry, and display blocked constraints/reasons where available.

UI & State Management Issues

U-1. Choosing a past schedule time silently changes the user's intent

- **Severity:** Medium
- **Location:** `app/src/main/kotlin/com/messages/app/ui/chat/ChatScreen.kt:2179`,
`app/src/main/kotlin/com/messages/app/ui/chat/ChatScreen.kt:2191`
- **What is wrong:** Any selected time in the past is coerced to exactly one minute from now rather than rejected or confirmed.
- **How it fails:** The user accidentally selects yesterday at 09:00 and taps Schedule. The app queues the message for one minute later, potentially sending an unintended message instead of explaining the invalid choice.
- **Suggested fix:** Disable confirmation for past instants, show a validation error, and require the user to select a future time; handle DST gaps/overlaps explicitly.

U-2. Enabling hidden previews leaves existing notifications exposed

- **Severity:** Medium
- **Location:**
`app/src/main/kotlin/com/messages/app/ui/settings/SettingsScreen.kt:140`,
`app/src/main/kotlin/com/messages/app/notify/MessageNotifier.kt:56`
- **What is wrong:** The setting refreshes widgets only. Notification content is decided when originally posted, and existing notifications are neither cancelled nor rebuilt.
- **How it fails:** Sensitive sender/body notifications are already on the lock screen; the user enables “hide previews,” but those existing notifications continue exposing content until dismissed or replaced.
- **Suggested fix:** On privacy-toggle change, cancel/repost every active message/fraud/reminder notification with redacted content, and make notification rendering derive from a centralized current privacy policy.

U-3. Blocking is based on exact raw number spelling

- **Severity:** Medium

- **Location:**

`app/src/main/kotlin/com/messages/app/ui/contact/ContactDetailScreen.kt:98,`
`app/src/main/kotlin/com/messages/app/ui/contact/ContactDetailScreen.kt:103,`
`app/src/main/kotlin/com/messages/app/ui/contact/ContactDetailScreen.kt:159`

- **What is wrong:** The created block rule and UI check compare raw strings case-insensitively but do not canonicalize national/international/punctuation variants.
- **How it fails:** Block `+81 90-1234-5678`; a later message arrives as `09012345678`. It does not match, is classified/notified normally, and the detail screen can claim the equivalent sender is unblocked.
- **Suggested fix:** Store a canonical E.164/keyed recipient identity with region/subscription context, retain the display form separately, and migrate legacy sender rules.

U-4. The Contact Detail “Lock chat” row is clickable but does nothing

- **Severity:** Medium

- **Location:**

`app/src/main/kotlin/com/messages/app/ui/contact/ContactDetailScreen.kt:342,`
`app/src/main/kotlin/com/messages/app/ui/contact/ContactDetailScreen.kt:346,`
`app/src/main/kotlin/com/messages/app/ui/contact/ContactDetailScreen.kt:452`

- **What is wrong:** A full clickable `DetailRow` is rendered with `onClick = {}` despite copy saying it points users to locked-space behavior.
- **How it fails:** The user taps “Lock chat”; there is no navigation, explanation, or state change, so they may believe a sensitive conversation was protected when it remains normal.
- **Suggested fix:** Wire it to the authenticated move/setup flow, or make it non-clickable explanatory text with a real action button. Confirm completion and resulting storage consequences.

U-5. Home navigation/search/selection state is not process-restorable

- **Severity:** Low

- **Location:** `app/src/main/kotlin/com/messages/app/ui/home/HomeViewModel.kt:34,`
`app/src/main/kotlin/com/messages/app/ui/home/HomeViewModel.kt:175,`
`app/src/main/kotlin/com/messages/app/ui/home/HomeViewModel.kt:223,`
`app/src/main/kotlin/com/messages/app/ui/home/HomeScreen.kt:216`

- **What is wrong:** Folder, unread filter, search chips/text/label, selected threads, and screen-local search visibility use ordinary `MutableStateFlow/remember`, not `SavedStateHandle/rememberSaveable`.

- **How it fails:** Android kills the process while a user is searching or selecting threads; returning from Recents resets the folder/query/selection, losing context and making an intended bulk action impossible to resume.
- **Suggested fix:** Persist stable navigation/search/filter values in `SavedStateHandle` and save screen-local visibility; intentionally clear destructive multi-selection if restoration would be unsafe, with an explicit rationale.

Performance, Resource & Battery Issues

P-1. Drive multipart upload duplicates an already-large encrypted snapshot in memory

- **Severity:** High
- **Location:** `app/src/main/kotlin/com/messages/app/drive/DriveClient.kt:118`,
`app/src/main/kotlin/com/messages/app/drive/DriveClient.kt:129`,
`app/src/main/kotlin/com/messages/app/drive/DriveClient.kt:137`
- **What is wrong:** Upload receives the full blob `ByteArray`, copies it into `ByteArrayOutputStream`, then `toByteArray()` copies the complete multipart body again.
- **How it fails:** A 200–500 MiB snapshot can require the original blob plus the stream backing array plus final body (and growth copies), causing OOM before any network bytes are sent.
- **Suggested fix:** Use Drive resumable upload and stream metadata/body from file or source, with bounded chunks and restart tokens; never assemble multipart content in heap.

P-2. Restore applies conversation preferences quadratically

- **Severity:** Medium
- **Location:** `core-messaging/src/main/kotlin/com/messages/core/backup/BackupManager.kt:805`,
`core-messaging/src/main/kotlin/com/messages/core/backup/BackupManager.kt:1022`
- **What is wrong:** For each normal and locked preference, restore reloads/scans `allConversations()`.
- **How it fails:** Restoring N conversation preferences performs N full database queries/list scans, approaching $O(N^2)$; a mailbox with thousands of conversations can take minutes and increase the chance of process death/partial restore.
- **Suggested fix:** Load one map keyed by canonical address/space or add indexed DAO lookup/batched update, then apply all preferences within the Room transaction.

P-3. Arbitrarily long archive passwords can exhaust UI and KDF memory

- **Severity:** Medium
- **Location:**
`app/src/main/kotlin/com/messages/app/ui/settings/SettingsScreen.kt:1408,`
`app/src/main/kotlin/com/messages/app/ui/settings/SettingsScreen.kt:1441,`
`app/src/main/kotlin/com/messages/app/ui/settings/SettingsScreen.kt:1481`
- **What is wrong:** Password and repeat fields have no maximum length before repeated String copies and conversion to `CharArray`/KDF input.
- **How it fails:** Pasting a multi-megabyte string into both fields causes Compose edits/recomposition, strength checks, copies, and PBKDF processing to allocate heavily or freeze/crash settings.
- **Suggested fix:** Enforce a documented grapheme/byte cap (for example 256 characters) in `onValueChange` and again at the crypto boundary; reject oversized restore secrets before KDF work.

P-4. Every locked-list emission decrypts every conversation preview

- **Severity:** Medium
- **Location:**
`app/src/main/kotlin/com/messages/app/ui/secret/LockedSpaceScreen.kt:78,`
`app/src/main/kotlin/com/messages/app/ui/secret/LockedSpaceScreen.kt:82,`
`app/src/main/kotlin/com/messages/app/ui/secret/LockedSpaceScreen.kt:84`
- **What is wrong:** The Room flow maps the complete category list through AES-GCM opening on each emission, even if only one row changed.
- **How it fails:** In a large locked inbox, a mute/unread/arrival update triggers decryption and allocation for every preview before the UI can render, increasing latency and power use.
- **Suggested fix:** Keep decrypted preview UI models in a session-scoped cache keyed by row ID+ciphertext, invalidate only changed rows, or query/decrypt paged visible data after authentication.

P-5. Contact search filters the complete address book on every keystroke

- **Severity:** Medium
- **Location:**
`app/src/main/kotlin/com/messages/app/ui/compose/NewMessageScreen.kt:82,`
`app/src/main/kotlin/com/messages/app/ui/compose/NewMessageScreen.kt:85,`

app/src/main/kotlin/com/messages/app/ui/compose/NewMessageScreen.kt:121

- **What is wrong:** `combine(allContacts, query)` immediately runs a full in-memory filter for every query update with no debounce/indexing and is collected on the `ViewModel/main` context.
- **How it fails:** On a large enterprise address book, fast typing repeatedly scans/allocates the entire list and causes visible input jank.
- **Suggested fix:** Debounce/distinct queries, normalize/index searchable fields once off-main, use `mapLatest` on `Dispatchers.Default`, and page provider results where possible.

P-6. MMS expiry pruning can orphan URI grants across a process crash

- **Severity:** Medium
- **Location:** `app/src/main/kotlin/com/messages/app/mms/MmsTransactions.kt:151`,
`app/src/main/kotlin/com/messages/app/mms/MmsTransactions.kt:159`,
`app/src/main/kotlin/com/messages/app/mms/MmsTransactions.kt:162`
- **What is wrong:** Stale registry records are durably removed before files are deleted and grants revoked.
- **How it fails:** The process dies after `editor.commit()` but before `settle`. On restart there is no record containing `contentUri`, so the orphan file sweep may delete the file but cannot revoke the explicit telephony URI grant.
- **Suggested fix:** Mark records `SETTLING`, revoke/delete, then remove only after success; retry incomplete settlement at startup.

P-7. Swipe handling launches a coroutine for every drag frame

- **Severity:** Low
- **Location:** `app/src/main/kotlin/com/messages/app/ui/home/HomeScreen.kt:1234`,
`app/src/main/kotlin/com/messages/app/ui/home/HomeScreen.kt:1237`,
`app/src/main/kotlin/com/messages/app/ui/home/HomeScreen.kt:1243`
- **What is wrong:** Each pointer delta starts a new coroutine merely to call `Animatable.snapTo`.
- **How it fails:** A fast swipe across many rows creates a burst of short coroutines that contend/cancel around one animation value, adding allocations and input latency.
- **Suggested fix:** Consume drag deltas within one gesture coroutine (`drag/snapTo`) or use `anchoredDraggable`, reserving separate launches for final settle animation only.

Accessibility Issues

A-1. The invisible PIN text field exposes the raw credential to semantics

- **Severity:** High
- **Location:** `app/src/main/kotlin/com/messages/app/ui/secret/SecretInputs.kt:155`,
`app/src/main/kotlin/com/messages/app/ui/secret/SecretInputs.kt:162`,
`app/src/main/kotlin/com/messages/app/ui/secret/SecretInputs.kt:167`
- **What is wrong:** The `BasicTextField` makes glyph color transparent but does not apply `PasswordVisualTransformation` or password semantics; visual hiding is not semantic hiding.
- **How it fails:** TalkBack or another accessibility service focusing the field can announce/read the underlying digit string even though sighted users see dots, disclosing the locked-space credential.
- **Suggested fix:** Apply password visual transformation and explicit password semantics while retaining an accessible label/count that never contains digits; verify with TalkBack and UI Automator accessibility-node inspection.

A-2. Settings switch rows create two independently actionable targets

- **Severity:** Medium
- **Location:**
`app/src/main/kotlin/com/messages/app/ui/settings/SettingsComponents.kt:120`,
`app/src/main/kotlin/com/messages/app/ui/settings/SettingsComponents.kt:128`,
`app/src/main/kotlin/com/messages/app/ui/settings/SettingsComponents.kt:131`
- **What is wrong:** The parent row is clickable and the trailing `Switch` also has `onCheckedChangeListener`, without merged/cleared child semantics.
- **How it fails:** TalkBack/switch access traverses two controls with the same purpose; activating both in sequence toggles on then off and creates confusing duplicate announcements.
- **Suggested fix:** Expose one merged switch-role node: make the child switch non-interactive (`onCheckedChangeListener=null`) and put role/state/action on the row, or make only the switch clickable with a properly associated label.

A-3. Locked conversation long-press action has no accessible label

- **Severity:** Low
- **Location:**
`app/src/main/kotlin/com/messages/app/ui/secret/LockedSpaceScreen.kt:297`,
`app/src/main/kotlin/com/messages/app/ui/secret/LockedSpaceScreen.kt:305`
- **What is wrong:** `combinedClickable` supplies `onLongClick` but no `onLongClickLabel` or equivalent custom accessibility action.

- **How it fails:** TalkBack users hear only the normal open action and cannot discover that a long press opens unlock/mute actions, making those controls effectively hidden.
- **Suggested fix:** Add a localized long-click label/custom action and validate traversal/action discovery with TalkBack, keyboard, and switch access.

Internationalisation & Localisation Issues

I-1. Core chat interactions still contain hardcoded English

- **Severity:** Medium
- **Location:** `app/src/main/kotlin/com/messages/app/ui/chat/ChatScreen.kt:1051,`
`app/src/main/kotlin/com/messages/app/ui/chat/ChatScreen.kt:1268,`
`app/src/main/kotlin/com/messages/app/ui/chat/ChatScreen.kt:1602,`
`app/src/main/kotlin/com/messages/app/ui/chat/ChatScreen.kt:1745,`
`app/src/main/kotlin/com/messages/app/ui/chat/ChatScreen.kt:1907,`
`app/src/main/kotlin/com/messages/app/ui/chat/ChatScreen.kt:1964,`
`app/src/main/kotlin/com/messages/app/ui/chat/ChatScreen.kt:2077,`
`app/src/main/kotlin/com/messages/app/ui/chat/ChatScreen.kt:2401`
- **What is wrong:** SIM labels, attachment choices, schedule presets, date terms, fraud warnings, message actions/status, filtered-message actions, and the entire message-info sheet bypass resources.
- **How it fails:** Change device language to Japanese: surrounding settings/resources may adapt, but high-risk fraud text and primary chat controls remain English; translators cannot change word order/plurals and TalkBack announces a mixed-language screen.
- **Suggested fix:** Move every user-facing string to resources, use formatted/plural resources rather than concatenation, and add a lint/source gate that allows literals only for non-UI constants/test tags.

I-2. The app ships no localized resource set

- **Severity:** Medium
- **Location:** `app/src/main/res/values/strings.xml:1`
- **What is wrong:** There are no values-`<locale>` string resources despite locale-sensitive messaging/fraud UX and a classifier containing Indic-language terms.
- **How it fails:** Every non-English user receives English onboarding, permissions rationale, privacy/security warnings, errors, and recovery instructions, including the credentials needed to regain backups.

- **Suggested fix:** Establish supported locales, begin with markets encoded by classifier policy plus the release market, professionally translate security/recovery copy, and add pseudo-locale/RTL CI screenshots.

I-3. Schedule picker forces 12-hour time for every locale/user setting

- **Severity:** Medium
- **Location:** `app/src/main/kotlin/com/messages/app/ui/chat/ChatScreen.kt:2140`, `app/src/main/kotlin/com/messages/app/ui/chat/ChatScreen.kt:2141`
- **What is wrong:** `rememberTimePickerState(is24Hour = false)` ignores `DateFormat.is24HourFormat`.
- **How it fails:** A user configured for 24-hour time must use AM/PM and can misread 18:00 as 06:00, scheduling at the wrong time.
- **Suggested fix:** Initialize from the system 12/24-hour preference and keep formatted confirmation consistent with it.

I-4. Today/Yesterday labels go stale and use a fixed 24-hour “day”

- **Severity:** Medium
- **Location:** `app/src/main/kotlin/com/messages/app/ui/chat/ChatScreen.kt:1613`, `app/src/main/kotlin/com/messages/app/ui/chat/ChatScreen.kt:1615`, `app/src/main/kotlin/com/messages/app/ui/chat/ChatScreen.kt:1621`
- **What is wrong:** `DatePill` remembers by timestamp only, hardcodes English, and computes yesterday as `now - 24h` rather than calendar-day arithmetic.
- **How it fails:** Leave chat open across midnight/time-zone change and labels do not update; around a DST transition, subtracting 24 hours can land on the wrong local date and label a message incorrectly.
- **Suggested fix:** Use localized resources and `java.time.LocalDate` in the current zone, key/recompute on date/zone/locale changes, and schedule a midnight refresh.

I-5. Pattern-progress plural selection uses the minimum, not the count

- **Severity:** Medium
- **Location:** `app/src/main/kotlin/com/messages/app/ui/secret/SecretInputs.kt:459`, `app/src/main/kotlin/com/messages/app/ui/secret/SecretInputs.kt:462`, `app/src/main/res/values/strings.xml:1221`

- **What is wrong:** `getQuantityString` chooses the grammatical form with `PATTERN_MIN_DOTS` while formatting the actual selected `dots` count.
- **How it fails:** With one selected dot and a four-dot minimum, languages whose one/few/many form depends on 1 receive the form for 4, so the live TalkBack progress sentence is grammatically wrong.
- **Suggested fix:** Pass `dots` as the quantity selector and keep the minimum only as a format argument; add plural tests for one/few/many locales.

I-6. Classifier and badge explanations are hardcoded English domain data

- **Severity:** Medium
- **Location:**
`protection-engine/src/main/kotlin/com/messages/protection/ProtectionEngine.kt:68,`
`protection-engine/src/main/kotlin/com/messages/protection/ProtectionEngine.kt:145,`
`protection-engine/src/main/kotlin/com/messages/protection/ProtectionEngine.kt:270,`
`protection-engine/src/main/kotlin/com/messages/protection/SenderBadges.kt:69`
- **What is wrong:** The pure engine returns English sentences rather than stable reason codes/parameters, coupling classification to one display language.
- **How it fails:** A Japanese UI receives English “Why filtered?” reasons and verified-badge explanations that cannot be translated or grammatically reordered without parsing prose.
- **Suggested fix:** Return stable typed reason codes with structured parameters (domain, amount, sender class); resolve localized copy in the app module and retain codes in backups/analytics rather than sentences.

API & Platform Compliance Issues

M-1. The debug harness's documented adb entry point is not actually callable as described

- **Severity:** Low
- **Location:** `app/src/debug/AndroidManifest.xml:4,`
`app/src/debug/AndroidManifest.xml:9,` `app/src/debug/AndroidManifest.xml:18`
- **What is wrong:** Comments say ordinary adb/shell broadcasts can reach the debug receivers, but a custom `signature` permission is enforced; shell does not automatically share the debug APK signing certificate or hold arbitrary signature permissions.

- **How it fails:** A developer follows the documented `adb shell am broadcast` harness flow on a normal device and receives a permission denial before the second token gate is evaluated, so device-level intake testing cannot run.
- **Suggested fix:** Decide the intended threat model: use a non-exported instrumentation/shell-supported test mechanism, or a debug-only permission level/gate that `adb` can genuinely satisfy. Add a CI/device smoke test and correct the comments.

Build, CI & Supply Chain Issues

B-1. Sensitive configuration remains reachable in Git history

- **Severity:** High
- **Location:** `PROGRESS.md:3`, `PROGRESS.md:5`, `PROGRESS.md:74`, `docs/ops/DISTRIBUTION_CHECKLIST.md:131`, `docs/ops/DISTRIBUTION_CHECKLIST.md:138`
- **What is wrong:** Current documentation confirms that prior commits/objects still contain sensitive personal/configuration material and that no history rewrite has occurred. The historical tree includes a `google-services.json`; its contents are intentionally not reproduced here.
- **How it fails:** Publishing, mirroring, or sharing a clone transfers reachable historical objects even though the working tree is redacted, exposing the old configuration/PII to every recipient and automated indexer.
- **Suggested fix:** Keep the repository private; inventory and rotate/revoke affected credentials, coordinate `git filter-repo/force-push`, invalidate old clones/forks, then expire reflogs/prune and verify all refs/objects from a fresh clone before distribution.

B-2. Advisory acceptances ignore the vulnerable version when matching coordinates

- **Severity:** Medium
- **Location:** `scripts/scan-dependencies.py:279`, `scripts/scan-dependencies.py:287`, `gradle/accepted-advisories.json:27`
- **What is wrong:** `covers` strips the version with `rsplit` and applies broad wildcard coordinates such as `io.netty:*`; the documented rationale is version/toolchain-specific.
- **How it fails:** The same exact advisory appears on another version/module in the accepted family after a dependency change. It is silently suppressed until expiry even if the new occurrence is shipped or has different reachability assumptions within the accepted scope.

- **Suggested fix:** Match exact normalized group, artifact, version range, scope, and dependency path; require re-review on any version/path change and keep wildcard module acceptances narrowly enumerated.

B-3. Release CI does not assert the intended version code

- **Severity:** Medium
- **Location:** `scripts/verify-release-artifact.sh:186`, `scripts/verify-release-artifact.sh:190`, `.github/workflows/ci.yml:243`
- **What is wrong:** Artifact verification checks version only when `EXPECTED_VERSION_CODE` is set, but the signed release job supplies only the certificate fingerprint.
- **How it fails:** A tag can build/sign/upload an APK with an accidental stale or wrong version code while the verifier prints package data and passes.
- **Suggested fix:** Derive the expected version from the tag/release metadata, pass it in every release job, and fail if the variable is absent for a tagged build.

B-4. `versionCode = 1` blocks the next Play update if version 1 is already published

- **Severity:** Medium
- **Location:** `app/build.gradle.kts:93`, `app/build.gradle.kts:97`
- **What is wrong:** Production identity is fixed at the initial version with no monotonic release mechanism.
- **How it fails:** After version code 1 reaches Play/internal distribution, a second build with code 1 is rejected as non-upgradable, delaying security fixes.
- **Suggested fix:** Source a monotonically increasing version code from versioned release metadata/CI, validate it against the tag and prior release, and keep local builds deterministic.

B-5. The pinned Android/toolchain set is materially stale and includes accepted known advisories

- **Severity:** Medium
- **Location:** `gradle/libs.versions.toml:2`, `gradle/libs.versions.toml:7`, `gradle/libs.versions.toml:8`, `gradle/accepted-advisories.json:43`, `gradle/accepted-advisories.json:71`
- **What is wrong:** Core pins remain 2024-era (AGP 8.5.2, Compose 2024.09, Room 2.6.1, etc.) in August 2026, while the repository explicitly accepts several vulnerable build dependencies; at least

one acceptance notes a newer AGP resolves the affected artifact.

- **How it fails:** Security/compatibility fixes in current Android libraries are absent, and accepted build-path vulnerabilities persist longer than necessary; a future SDK/Play toolchain shift becomes a large risky upgrade instead of routine maintenance.
- **Suggested fix:** Establish monthly Renovate/Dependabot update PRs, upgrade AGP/Kotlin/Compose/Room in small verified steps, rerun OSV/checksum generation, and time-box every acceptance to an owned upgrade issue.

B-6. Repository ref/object health is intentionally unresolved

- **Severity:** Low
- **Location:** docs/ops/DISTRIBUTION_CHECKLIST.md:131, docs/ops/DISTRIBUTION_CHECKLIST.md:135, docs/ops/DISTRIBUTION_CHECKLIST.md:145
- **What is wrong:** The repository records an invalid remote HEAD and unreachable/dangling objects, deferred until the sensitive-history rewrite.
- **How it fails:** Clone/default-branch tooling can behave inconsistently and old objects remain discoverable locally; release provenance/audits can accidentally include unintended history.
- **Suggested fix:** After the authorized history rewrite, repair remote HEAD, expire/prune unreachable objects, run `git fsck --full`, and verify a fresh minimal clone before releasing source.

Test Coverage Gaps

V-1. There is no instrumented/device test suite for Android-critical behavior

- **Severity:** High
- **Location:** app/build.gradle.kts:168, core-messaging/build.gradle.kts:20, .github/workflows/ci.yml:73, .github/workflows/ci.yml:87
- **What is wrong:** Dependencies and CI run JVM/Robolectric tests only; there is no `androidTest/managed-device` task for real Telephony provider, SmsManager callbacks, Keystore, FileProvider grants, PendingIntent identity, WorkManager/Doze, notification privacy, or Compose accessibility.
- **How it fails:** OEM/framework differences such as mutable callback fill-in behavior, provider row counts, Keystore invalidation, TalkBack PIN semantics, or concurrent receiver lifecycle can regress while every CI test remains green.

- **Suggested fix:** Add Gradle Managed Device/instrumented suites for the four mandatory SMS components, provider CRUD/rollback, real Keystore envelopes, PendingIntent generations, notifications, backup process death, and Compose semantics; run a smaller physical multi-SIM matrix before release.

V-2. Migration tests omit every committed schema from v5 through v10 and miss FTS behavior

- **Severity:** High
- **Location:**
`core-messaging/src/test/kotlin/com/messages/core/db/MigrationFromV1Test.kt:19,`
`core-messaging/src/test/kotlin/com/messages/core/db/MigrationFromV1Test.kt:120,`
`core-messaging/src/test/kotlin/com/messages/core/db/MigrationFromV1Test.kt:243`
- **What is wrong:** Tests hand-build only v1–v4 and open through the whole chain; they never use committed v5–v10 schema JSONs as starting points or assert FTS insert/update/delete triggers.
- **How it fails:** The missing 5→6 FTS triggers pass because final Room schema validation does not prove runtime synchronization behavior, and a defect affecting only a v7/v8/v9 starting schema can ship untested.
- **Suggested fix:** Use `MigrationTestHelper` with every exported schema 5–10, seed version-specific state, migrate one edge and all paths, then behaviorally assert FTS/provider mappings/locked-space/send attempts.

V-3. Several app “tests” assert source spelling rather than behavior

- **Severity:** Medium
- **Location:** `app/src/test/kotlin/com/messages/app/ui/HardcodedStringTest.kt:17,`
`app/src/test/kotlin/com/messages/app/ui/HardcodedStringTest.kt:115,`
`app/src/test/kotlin/com/messages/app/ui/HardcodedStringTest.kt:182,`
`app/src/test/kotlin/com/messages/app/ui/LifecycleAwareCollectionTest.kt:32`
- **What is wrong:** Regex/text scans infer localization and lifecycle safety from one-line syntax; they neither compile semantic intent nor run the UI/lifecycle.
- **How it fails:** Hardcoded strings assembled via variables/multiline helpers (as current chat code does) evade the patterns, and an aliased/custom lifecycle-unaware collector can pass. Conversely, a behavior-preserving formatting/refactor can fail the lexical test.
- **Suggested fix:** Keep lightweight lint guards, but add Android Lint/UAST rules plus runtime Compose tests that switch locale/background state and assert rendered resources and stopped collection.

V-4. The corpus gate has no representative non-Latin scripts and tolerates genuine Review results

- **Severity:** Medium
- **Location:**
`protection-engine/src/test/kotlin/com/messages/protection/CorpusRegressionTest.kt:9,`
`protection-engine/src/test/kotlin/com/messages/protection/CorpusRegressionTest.kt:66,`
`protection-engine/src/test/resources/corpus.json:1`
- **What is wrong:** The 513-entry corpus contains no Japanese/Han, Arabic, or Devanagari messages, and the genuine gate rejects only Spam/Blocked, allowing a meaning-changing shift of all genuine messages into Review.
- **How it fails:** Locale/script-specific normalization and false positives can regress unnoticed; a refactor that sends every genuine personal message to Review still passes gate 4 despite materially degrading the inbox.
- **Suggested fix:** Build stratified, privacy-safe corpora by locale/script/sender type and adversarial family; set per-class confusion-matrix ceilings including Review, and report confidence intervals rather than one aggregate rate.

V-5. The design system has no tests

- **Severity:** Low
- **Location:** `design-system/build.gradle.kts:19,`
`design-system/src/main/kotlin/com/messages/designsystem/Theme.kt:37`
- **What is wrong:** The module declares only runtime/debug dependencies and has no unit, screenshot, contrast, dynamic-type, or RTL regression suite.
- **How it fails:** A palette/typography/motion change can reduce contrast or clip large text across every screen without a module-level signal; app lexical tests do not exercise rendering.
- **Suggested fix:** Add contrast/property tests for every foreground/background pair, screenshot tests across light/dark/dynamic seeds and font scales, RTL/pseudo-locale previews, and reduced-motion checks.

Privacy & Compliance Issues

O-1. The privacy policy is not publishable and blocks a compliant store submission

- **Severity:** High

- **Location:** docs/ops/privacy_policy.md:3, docs/ops/privacy_policy.md:5, docs/ops/privacy_policy.md:8
- **What is wrong:** Effective date, accountable operator, and monitored contact remain literal placeholders; the document itself says stores will reject it.
- **How it fails:** Submit this default-SMS app with restricted SMS permissions and the current policy URL/content. Reviewers/users cannot identify or contact a controller, creating immediate Play policy/legal rejection exposure.
- **Suggested fix:** Fill verified legal identity/date/contact, publish at a stable public HTTPS URL, align the Play Data safety/SMS declaration with actual optional Drive/link-preview/carrier flows, and establish an owner/review date for every release.

O-2. The policy overclaims locked-space at-rest protection

- **Severity:** High
- **Location:** docs/ops/privacy_policy.md:32, docs/ops/privacy_policy.md:34, docs/ops/privacy_policy.md:36, core-messaging/src/main/kotlin/com/messages/core/secret/LockedContent.kt:69, core-messaging/src/main/kotlin/com/messages/core/secret/LockedContent.kt:61
- **What is wrong:** Policy says locked words are encrypted and unreadable from an image/other app, but implementation can store plaintext on key/cipher failure, leaves attachments plaintext, and trash restore can republish plaintext to Telephony.
- **How it fails:** A user relies on the policy and locks a sensitive chat during Keystore failure or with media; a forensic extraction/default-app switch reveals content the policy explicitly says is protected.
- **Suggested fix:** Fix S-1/S-3/D-1 before making the claim. Until then, disclose fail-open and attachment/provider exceptions prominently in policy and setup UI, with version/effective-date change notice.

O-3. The policy attributes biometric protection to a knowledge-only locked space

- **Severity:** Medium
- **Location:** docs/ops/privacy_policy.md:52, docs/ops/privacy_policy.md:58, app/src/main/res/values/strings.xml:1121
- **What is wrong:** Permission table says Biometric is for “the optional app lock and the locked space,” while setup explicitly says locked chats use only the secret code, not fingerprint/phone lock.

- **How it fails:** Users infer hardware/biometric gating for locked-space key use and choose a weaker PIN under a false threat model; Keystore actually requires no biometric authentication.
- **Suggested fix:** State that biometric applies only to the outer app lock, and describe the locked space's knowledge factor/software-key model consistently.

O-4. Restore confirmation falsely promises that nothing is overwritten

- **Severity:** Medium
- **Location:** `app/src/main/res/values/strings.xml:509`,
`core-messaging/src/main/kotlin/com/messages/core/backup/BackupManager.kt:670`,
`core-messaging/src/main/kotlin/com/messages/core/backup/BackupManager.kt:805`
- **What is wrong:** User copy says data/settings are added and nothing on-device is overwritten, but sensitivity, OTP cleanup, preview setting, and conversation preferences are explicitly replaced.
- **How it fails:** A user restores without recording current preferences because of the promise; their security/privacy/notification behavior changes immediately and rollback is unavailable after success.
- **Suggested fix:** Enumerate exactly what merges, replaces, skips, and cannot be represented; show a preflight diff and allow category-specific restore/undo.

O-5. Local backup copy falsely says the file stays on-device

- **Severity:** Medium
- **Location:** `app/src/main/res/values/strings.xml:559`,
`app/src/main/res/values/strings.xml:560`,
`app/src/main/kotlin/com/messages/app/ui/settings/SettingsScreen.kt:889`,
`app/src/main/kotlin/com/messages/app/ui/settings/SettingsScreen.kt:918`
- **What is wrong:** Copy says “Everything stays on this device,” but Storage Access Framework lets the user select cloud-backed document providers, USB, network storage, or synced folders.
- **How it fails:** A user chooses a Drive/Dropbox document provider believing the statement and exports the archive off-device; confidentiality then depends entirely on their password strength.
- **Suggested fix:** Say “saved wherever you choose” and explain that cloud/document-provider handling is outside the app; show destination authority and encryption/password warning before writing.

O-6. Locked-space settings contradict the setup disclosure and current code

- **Severity:** Medium

- **Location:** `app/src/main/res/values/strings.xml:1121`,
`app/src/main/res/values/strings.xml:1124`,
`app/src/main/res/values/strings.xml:1171`,
`app/src/main/res/values/strings.xml:1178`
- **What is wrong:** Setup says provider copies are deleted and words encrypted, while the later About text says SMS content still lives in shared storage and locked chats only hide it in-app.
- **How it fails:** The same user sees mutually exclusive security claims and cannot make an informed decision about switching default SMS apps, device extraction, or backup.
- **Suggested fix:** Maintain one versioned, tested locked-space disclosure component reused by setup/settings/policy; generate copy from explicit capability flags if degradation is possible.

O-7. Drive introduction presents account-held custody as universally true

- **Severity:** Medium
- **Location:** `app/src/main/res/values/strings.xml:879`,
`app/src/main/res/values/strings.xml:880`,
`app/src/main/res/values/strings.xml:1052`
- **What is wrong:** The unconditional intro says the Google account is the key and no password is needed, even when the same screen supports/uses user-held recovery-code/password custody.
- **How it fails:** A user in user-held mode believes account sign-in alone will restore, fails to preserve the recovery secret, and loses access after device loss.
- **Suggested fix:** Render custody-specific introductory copy from authoritative remote/key state and never show account-only recovery language in user-held mode.

O-8. “Forever” and “nothing ever deleted” promises conflict with enabled deletion features

- **Severity:** Medium
- **Location:** `app/src/main/res/values/strings.xml:254`,
`app/src/main/res/values/strings.xml:265`, `app/src/main/res/values/strings.xml:549`,
`app/src/main/res/values/strings.xml:551`,
`app/src/main/res/values/strings.xml:1165`
- **What is wrong:** Onboarding makes an unconditional permanence promise while OTP cleanup, spam cleanup/trash purge, manual delete, and secret reset remove messages.
- **How it fails:** A user enables cleanup or resets locked space after relying on “Every filtered message stays reviewable, forever”; the data is permanently purged contrary to the onboarding

guarantee.

- **Suggested fix:** Replace absolutes with precise defaults and exceptions: filtering itself never deletes, while user-enabled retention/reset actions do; link directly to retention controls.

O-9. Product/specification documents contradict actual outbound-data and locked-storage behavior

- **Severity:** Low
- **Location:** PRD_Messages.md:321, PRD_Messages.md:325, PROJECT_HANDOFF.md:6, PROJECT_HANDOFF.md:8, README.md:40, README.md:43
- **What is wrong:** PRD/handoff promise message content never leaves device despite optional Drive backup/link previews/carrier reporting; README says locked SMS remains in Telephony although current code purges it.
- **How it fails:** A future developer/reviewer/store submission built from these “authoritative” documents can reintroduce the wrong behavior or publish an inaccurate Data safety/SMS declaration.
- **Suggested fix:** Mark superseded documents, link one living architecture/privacy source of truth, add doc assertions for externally testable claims, and review it with every data-flow change.

Code Quality & Maintainability

Q-1. MessageRepository is a 2,000-line cross-domain God object

- **Severity:** Low
- **Location:**
core-messaging/src/main/kotlin/com/messages/core/MessageRepository.kt:173,
core-messaging/src/main/kotlin/com/messages/core/MessageRepository.kt:420,
core-messaging/src/main/kotlin/com/messages/core/MessageRepository.kt:708,
core-messaging/src/main/kotlin/com/messages/core/MessageRepository.kt:906,
core-messaging/src/main/kotlin/com/messages/core/MessageRepository.kt:1475,
core-messaging/src/main/kotlin/com/messages/core/MessageRepository.kt:1967
- **What is wrong:** One singleton owns SMS/MMS intake, provider mirroring, classification, contacts/threading, send state, secret-space crypto/moves, trash, and reputation.
- **How it fails:** A change to provider ordering or space routing crosses unrelated methods and transaction boundaries; defects such as restore/provider leaks and lost updates are hard to isolate or unit-test without Android/Room global state.

- **Suggested fix:** Split bounded services (`IntakeCoordinator`, `ProviderMirror`, `SendCoordinator`, `LockedStore`, `TrashService`, `ReputationStore`) behind narrow interfaces and make transaction ownership explicit.

Q-2. Backup restore is a monolithic four-store transaction simulator

- **Severity:** Low
- **Location:**
`core-messaging/src/main/kotlin/com/messages/core/backup/BackupManager.kt:650`,
`core-messaging/src/main/kotlin/com/messages/core/backup/BackupManager.kt:659`,
`core-messaging/src/main/kotlin/com/messages/core/backup/BackupManager.kt:848`,
`core-messaging/src/main/kotlin/com/messages/core/backup/BackupManager.kt:861`
- **What is wrong:** A single 1,100-line object parses, validates, mutates preferences, Room, Telephony, files, locked envelopes, dedupe, and manual rollback.
- **How it fails:** Adding one backed-up field requires edits across validation/export/import/undo/rebuild, and omission from any step produces partial-state bugs that compile and often pass happy-path tests.
- **Suggested fix:** Define a versioned manifest and staged restore plan with per-store adapters, immutable preflight, durable journal, and explicit commit/reconcile phases.

Q-3. Primary UI files are too large to reason about or recompose confidently

- **Severity:** Low
- **Location:** `app/src/main/kotlin/com/messages/app/ui/chat/ChatScreen.kt:1`,
`app/src/main/kotlin/com/messages/app/ui/chat/ChatScreen.kt:2437`,
`app/src/main/kotlin/com/messages/app/ui/settings/SettingsScreen.kt:1`,
`app/src/main/kotlin/com/messages/app/ui/settings/SettingsScreen.kt:1496`
- **What is wrong:** Chat and Settings combine navigation state, launchers, domain decisions, dialogs, formatting, security-sensitive clipboard/report actions, and many composables in 2,437/1,496-line files.
- **How it fails:** Small feature edits invalidate broad source tests/review scope, hidden hardcoded strings and inconsistent helpers slip through, and unstable state captured high in the tree can trigger unnecessarily broad recomposition.
- **Suggested fix:** Extract feature-state holders and focused UI components/files with stable immutable parameters; preview/test dialogs/cards/actions independently and keep domain work out of composition.

Q-4. Persistent domain state is stringly typed

- **Severity:** Low
- **Location:** `core-messaging/src/main/kotlin/com/messages/core/db/Entities.kt:65`,
`core-messaging/src/main/kotlin/com/messages/core/db/Entities.kt:69`,
`core-messaging/src/main/kotlin/com/messages/core/db/Entities.kt:82`,
`core-messaging/src/main/kotlin/com/messages/core/db/Entities.kt:113`
- **What is wrong:** Category, protected label, send status, and space are unrestricted strings duplicated in SQL/UI/backup code.
- **How it fails:** A typo/new enum value compiles and persists; fixed folder/status branches do not recognize it, causing messages to disappear from expected queries or remain in an unhandled status until ad-hoc fallback code is added.
- **Suggested fix:** Use centrally serialized enums/value classes with explicit unknown handling and Room converters/check constraints; version conversions at backup/migration boundaries.

Q-5. Locked crypto's public API accepts arbitrary AAD strings

- **Severity:** Low
- **Location:**
`core-messaging/src/main/kotlin/com/messages/core/secret/LockedContent.kt:120`,
`core-messaging/src/main/kotlin/com/messages/core/secret/LockedContent.kt:139`
- **What is wrong:** Callers pass a raw field: `String` even though only three exact values are valid, inviting typo/domain mismatch.
- **How it fails:** New code seals with `"Body"` and opens with `"body"`; GCM correctly rejects it and `openText` returns an opaque blob, with no compile-time or typed error.
- **Suggested fix:** Make low-level methods private/internal and expose typed `sealBody/openBody`, or a closed field enum plus structured result; never return ciphertext as if it were plaintext success.

Q-6. Backup DTO mapping relies on a long positional constructor

- **Severity:** Low
- **Location:**
`core-messaging/src/main/kotlin/com/messages/core/backup/BackupManager.kt:52`,
`core-messaging/src/main/kotlin/com/messages/core/backup/BackupManager.kt:265`
- **What is wrong:** `toBackupMessage` supplies many same-typed fields positionally, so field additions/reordering are easy to mis-map while compiling.

- **How it fails:** Swapping two Boolean/String positions during schema evolution silently writes the wrong semantics into every backup and is discovered only during restore.
- **Suggested fix:** Use named arguments, versioned DTO conversion tests/golden fixtures, and explicit defaults for every added field.

Suggestions & Improvements

G-1. Add a durable cross-store integrity reconciler

- **Severity:** Low
- **Location:** `core-messaging/src/main/kotlin/com/messages/core/db/Entities.kt:177`, `core-messaging/src/main/kotlin/com/messages/core/backup/BackupManager.kt:659`
- **What is wrong:** Improvement opportunity: provider rows, Room index rows, attachment files, work requests, and key/envelope state have local cleanup paths but no single observable invariant report/reconciler.
- **How it fails:** A process dies between stores and the app can retain orphan grants/files, missing provider mappings, or scheduled rows without work until a user encounters a symptom.
- **Suggested fix:** Maintain durable outbox/journal states and run a bounded startup/maintenance reconciler that reports and repairs orphan/missing relationships idempotently, without logging PII.

G-2. Centralize phone/sender identity normalization

- **Severity:** Low
- **Location:** `core-messaging/src/main/kotlin/com/messages/core/MessageRepository.kt:700`, `app/src/main/kotlin/com/messages/app/ui/contact/ContactDetailScreen.kt:103`, `app/src/main/kotlin/com/messages/app/ui/compose/NewMessageScreen.kt:183`
- **What is wrong:** Improvement opportunity: threading, group uniqueness, block rules, reputation, contacts, and dedupe each normalize addresses differently.
- **How it fails:** The same person can acquire separate thread/reputation/block identities or receive duplicate group sends depending on `+country`, national prefix, punctuation, and SIM region.
- **Suggested fix:** Create one versioned `RecipientIdentity` service using libphonenumber plus subscription region, with raw display value, canonical key, confidence, and migration aliases consumed everywhere.

G-3. Give users an explicit backup health/coverage receipt

- **Severity:** Low
- **Location:** `app/src/main/kotlin/com/messages/app/drive/DriveBackup.kt:637`,
`core-messaging/src/main/kotlin/com/messages/core/backup/BackupManager.kt:198`,
`core-messaging/src/main/kotlin/com/messages/core/backup/BackupManager.kt:294`
- **What is wrong:** Improvement opportunity: current status is time/size/count, not a verifiable statement of which categories, locked rows, media, and settings were authenticated and restorable.
- **How it fails:** Silent omissions remain invisible until disaster recovery, when the source device may no longer exist.
- **Suggested fix:** Store and display an authenticated manifest/receipt with counts, bytes, digests, omissions, custody/account ID, and a periodic local decrypt/parse self-test; never prune the last verified-complete snapshot.

G-4. Add adversarial state-machine and fault-injection tests

- **Severity:** Low
- **Location:** `app/src/main/kotlin/com/messages/app/schedule/Scheduling.kt:35`,
`core-messaging/src/main/kotlin/com/messages/core/backup/BackupManager.kt:848`,
`core-messaging/src/main/kotlin/com/messages/core/MessageRepository.kt:1500`
- **What is wrong:** Improvement opportunity: the hardest behavior is event ordering/process death, but tests emphasize pure/JVM happy paths and source guards.
- **How it fails:** Rare interleavings—callback after resend, death midway through space move/restore, two receivers, account switch—ship without a deterministic reproducer.
- **Suggested fix:** Model send/restore/space-move states explicitly and property-test every event ordering; inject failures after each durable/external side effect and assert invariants after restart.

G-5. Add privacy-preserving security-degradation diagnostics

- **Severity:** Low
- **Location:**
`core-messaging/src/main/kotlin/com/messages/core/secret/LockedContent.kt:113`,
`app/src/main/kotlin/com/messages/app/drive/DriveBackup.kt:82`
- **What is wrong:** Improvement opportunity: serious states such as locked encryption unavailable, omitted locked backup, stale grants, or account/key mismatch are mostly booleans/errors visible only at the moment of failure.

- **How it fails:** A user continues for days with plaintext locked arrivals or incomplete backups and support cannot distinguish it without inspecting sensitive data.
- **Suggested fix:** Maintain a local, PII-free security health ledger with timestamps/counts and actionable notifications; never log bodies/addresses and let users export redacted diagnostics.

Reviewed Controls With No Issues Found

No additional defects were found in the following reviewed controls; these should be preserved with regression tests:

- Mandatory exported SMS/MMS components are protected by system signature permissions, while all internal callback receivers/widgets are non-exported:
app/src/main/AndroidManifest.xml:67, app/src/main/AndroidManifest.xml:102, app/src/main/AndroidManifest.xml:138.
- FileProvider is non-exported and exposes only the two required cache subdirectories, not broad files/root paths: app/src/main/AndroidManifest.xml:162, app/src/main/res/xml/file_paths.xml:1.
- Android Auto Backup and device-to-device transfer are denied across supported versions: app/src/main/AndroidManifest.xml:54, app/src/main/res/xml/backup_rules.xml:13, app/src/main/res/xml/data_extraction_rules.xml:11.
- Release networking denies cleartext and trusts only system anchors; there is no permissive user-CA/debug override in main: app/src/main/res/xml/network_security_config.xml:57.
- Backup envelopes use fresh 96-bit GCM nonces and v2 authenticates the serialized header as AAD; the issues above are around bounds/lifecycle rather than nonce reuse:
core-messaging/src/main/kotlin/com/messages/core/backup/BackupCrypto.kt:41, core-messaging/src/main/kotlin/com/messages/core/backup/BackupCrypto.kt:220, core-messaging/src/main/kotlin/com/messages/core/backup/BackupCrypto.kt:358.
- Every schema edge 1→10 has an explicit migration; DB-1 is specifically the behavioral omission of FTS triggers, not fallback/destructive migration:
core-messaging/src/main/kotlin/com/messages/core/db/Migrations.kt:28, core-messaging/src/main/kotlin/com/messages/core/db/Migrations.kt:122.
- The design-system category palettes pair explicit container/foreground colors and Material typography supports scalable text; no concrete contrast defect was established statically:
design-system/src/main/kotlin/com/messages/designsystem/Theme.kt:37.
- CI actions are commit-SHA pinned, release signing is required/verified, dependency checksum verification and OSV/SBOM scanning are present: .github/workflows/ci.yml:9, .github/workflows/ci.yml:121, .github/workflows/ci.yml:174.

- Production flow collection uses lifecycle-aware Compose adapters in the reviewed screens; the remaining issue is that several flows lack typed loading/error states:

`app/src/test/kotlin/com/messages/app/ui/LifecycleAwareCollectionTest.kt:32.`

- Link previews are opt-in and the fetch path applies HTTPS, DNS/private-address, redirect, size, and content-type controls; no current SSRF/cleartext bypass was established:

`app/src/main/kotlin/com/messages/app/ui/chat/LinkPreview.kt:15,`

`app/src/main/kotlin/com/messages/app/net/SafeHttp.kt:31,`

`app/src/main/kotlin/com/messages/app/net/SafeHttp.kt:73.`

Summary

Category	Critical	High	Medium	Low	Total
Security	1	2	1	0	4
Cryptography & key management	0	1	3	0	4
Drive key custody	0	2	1	0	3
Data storage & privacy	0	1	2	1	4
Concurrency & threading	0	4	7	0	11
Messaging correctness & logic	0	7	11	0	18
Backup & restore	0	8	11	1	20
Database schema & migrations	0	1	0	0	1

Classifier, spam & fraud	0	4	7	1	12
Error handling	0	1	3	0	4
UI & state management	0	0	4	1	5
Performance, resources & battery	0	1	5	1	7
Accessibility	0	1	1	1	3
Internationalisation & localisation	0	0	6	0	6
API & platform compliance	0	0	0	1	1
Build, CI & supply chain	0	1	4	1	6
Test coverage	0	2	2	1	5
Privacy & compliance	0	2	6	1	9
Code quality & maintainability	0	0	0	6	6
Suggestions & improvements	0	0	0	5	5
Total	1	38	74	21	134

Five findings to fix first

1. **S-1 — Locked encryption fails open:** it directly defeats the flagship privacy guarantee and is explicitly enshrined by a regression test.
2. **S-3 — Locked trash restore republishes plaintext:** a normal UI action reverses Telephony-provider isolation without warning.
3. **R-1 — Restore rollback leaves damaged Room rows:** one failure can create an unrecoverable partial restore that retries refuse to repair.
4. **L-1 — Partial group send creates duplicates:** ordinary radio failure can resend already-delivered content to real recipients.
5. **R-2 — Drive state crosses accounts:** account switching can skip protection or reuse another account's key/checkpoint state.